UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

Apellidos y nombres: Pinto Caiza Diego Josuey

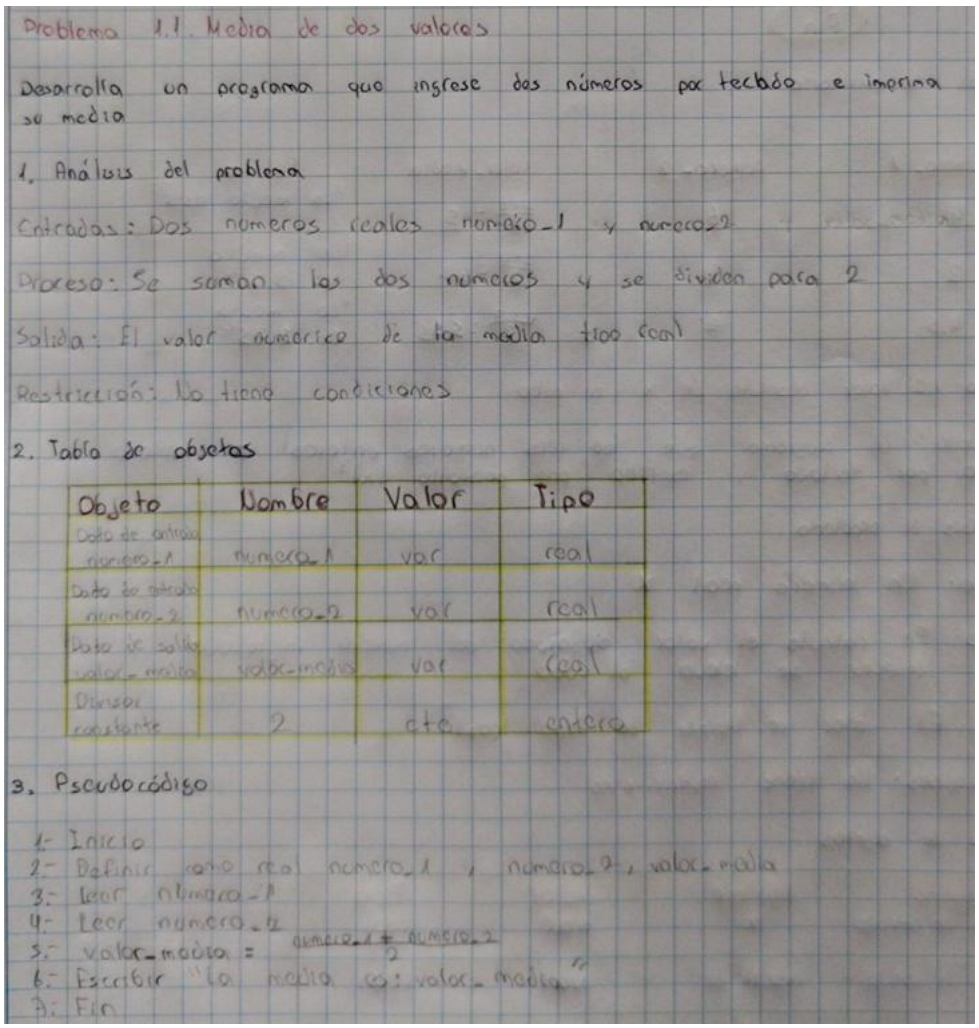
Docente: Ing. Ruiz Robalino Jenny Alexandra

Asignatura: Fundamentos de la programación

NRC: 36460

Fecha de entrega: 23/10/2026

Tema: Codebloks



Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media

1. Análisis del problema

Entradas: Dos numeros reales numero_1 y numero_2

Proceso: Se suman los dos numeros y se dividen para 2

Salida: El valor numerico de la media tipo real

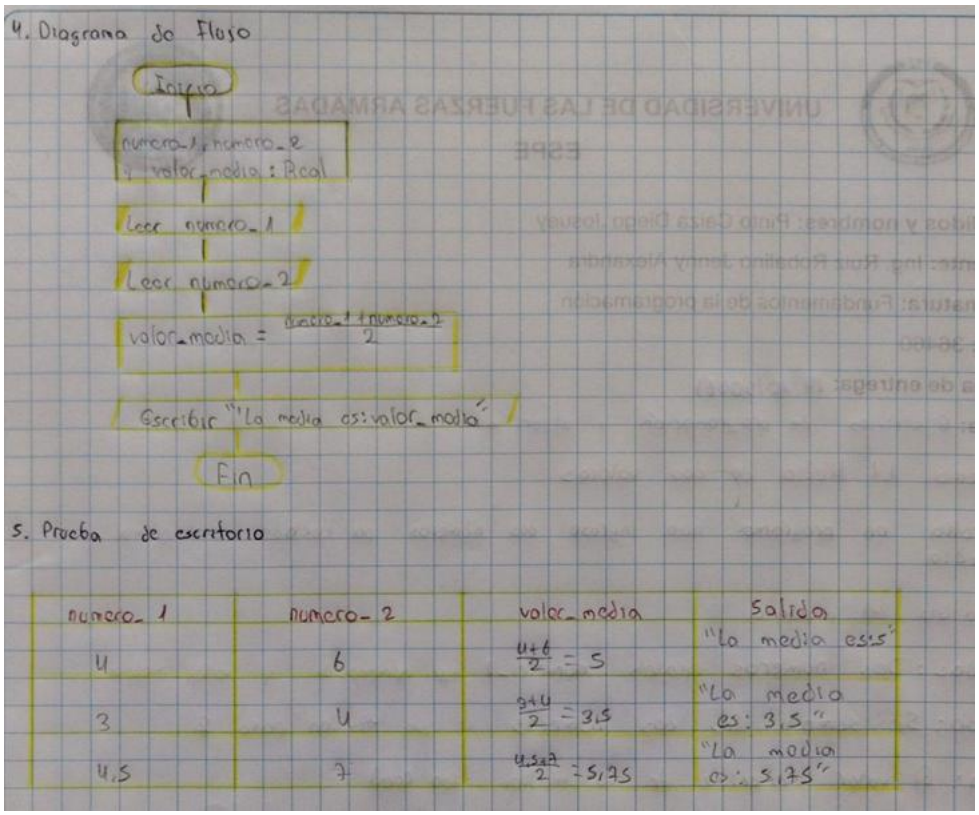
Restricción: No tiene condiciones

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada numero_1	numero_1	var	real
Dato de entrada numero_2	numero_2	var	real
Dato de salida valor_media	valor_media	var	real
Divisor constante	2	cte	entero

3. Pseudocódigo

1- Inicio
2- Definir como real numero_1 , numero_2, valor_media
3- Leer numero_1
4- Leer numero_2
5- valor_media = (numero_1 + numero_2) / 2
6- Escribir "la media es: valor_media"
7- Fin




```c
#include <stdio.h>

int main()
{
    float n1,n2,n3;
    int i;
    i=2;
    printf("Ingrese numero 1: ");
    scanf("%f",&n1);
    printf("Ingrese el numero 2: ");
    scanf("%f",&n2);
    n3= (n1+n2)/i;
    printf("El resultado final es: %.2f\n", n3);

}
```

```
Ingrese numero 1: 5
Ingrese el numero 2: 4
El resultado final es: 4.50

Process returned 0 (0x0)   execution time : 7.955 s
Press any key to continue.
```

Problema 1.2. Valor absolato de x al cubo

Desarrolla un programa que lea (dos numeros enteros) un numero (entero) por teclado y escriba por pantalla $|x|^3$

1. Analisis de problema

Entrada: Un numero real x

Proceso: Se calcula el valor absoluto del numero ingresado y despues se eleva al cubo

Salida: El valor resultante de tipo real

Restricciones: No tiene restricciones

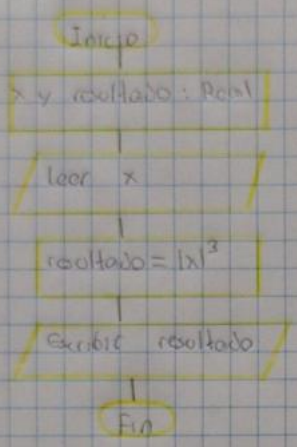
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	var	real
Dato de salida	resultado	var	real
Exponente constante	3	cte	entero

3. Pseudocodigo

1: Inicio
2: Definir x y resultado como real
3: Leer x
4: resultado = $|x|^3$
5: Escribir "El valor absoluto al cubo es resultado"
6: Fin

4. Diagrama de flujo



5. Prueba de escritorio

x	|x|	resultado	Salida
2	2	$2^3 = 8$	"El valor absoluto es 8"
-3	3	$3^3 = 27$	"El valor absoluto es 27"
0	0	$0^3 = 0$	"El valor absoluto es 0"

```c
#include <math.h>
#include <stdio.h>

int main()
{
    float n1,n3 ;
    printf("Ingrese numero 1: ");
    scanf("%f",&n1);
    n3= fabs(n1*n1*n1);
    printf("El resultado final es: %.2f\n", n3);

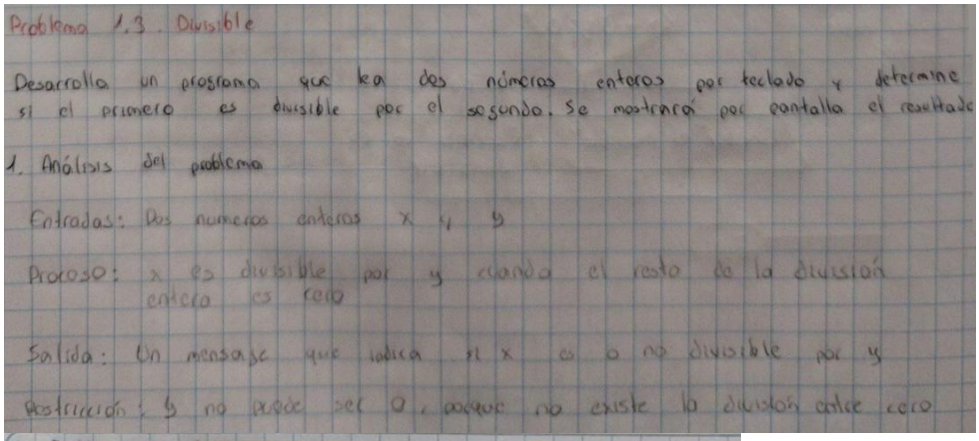
}
```

```
Ingrese numero 1: 4
El resultado final es: 64.00

Process returned 0 (0x0)   execution time : 2.262 s
Press any key to continue.
```

```
Ingrese numero 1: -4
El resultado final es: 64.00

Process returned 0 (0x0)   execution time : 3.265 s
Press any key to continue.
```



Problema 1.3. Divisible

Desarrolla un programa que lea dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado

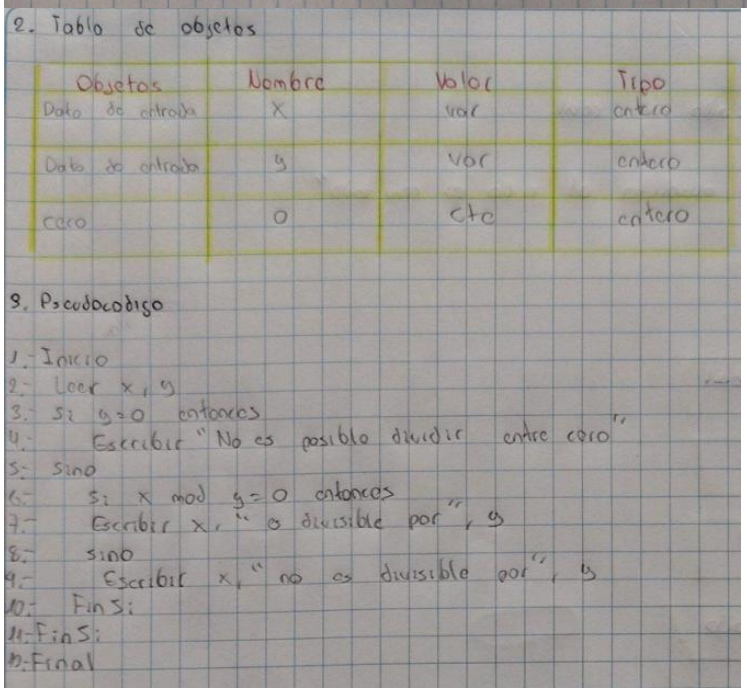
1. Análisis del problema

Entradas: Dos numeros enteros x y

Proceso: x es divisible por y cuando el resto de la división entera es cero

Salida: Un mensaje que indica si x es o no divisible por y

Restricción: y no puede ser 0, porque no existe la división entre cero

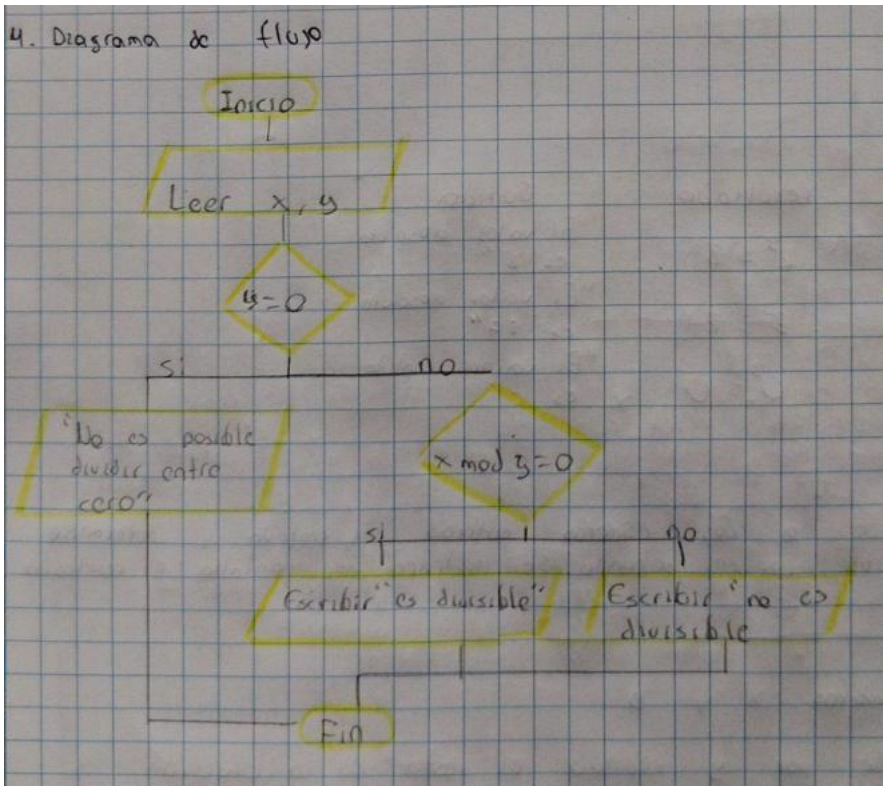


2. Tabla de objetos

Objetos	Nombre	Valor	Tipo
Dato de entrada	x	var	entero
Dato de entrada	y	var	entero
cero	0	cte	entero

3. Pseudocodigo

1.- Inicio
2.- Leer x, y
3.- Si y=0 entonces
4.- Escribir "No es posible dividir entre cero"
5.- Sino
6.- Si x mod y = 0 entonces
7.- Escribir x, " es divisible por ", y
8.- sino
9.- Escribir x, " no es divisible por ", y
10.- Fin Si
11.- Fin Si
12.- Final




```c
#include <stdio.h>

int main()
{
    int n1,n2;

    printf("Ingrese numero 1: ");
    scanf("%d",&n1);
    printf("Ingrese el numero 2: ");
    scanf("%d",&n2);
    if (n2==0)
        printf("No es posible dividir entre cero\n");
        else
        if(n1%n2==0)
            printf("%d es divisible por %d\n",n1,n2);
        else
            printf("%d no es divisible por %d\n",n1,n2);


}
```

```
Ingrese numero 1: 4
Ingrese el numero 2: 2
4 es divisible por 2

Process returned 0 (0x0)   execution time : 3.346 s
Press any key to continue.
```

```
Ingrese numero 1: 5
Ingrese el numero 2: 8
5 no es divisible por 8

Process returned 0 (0x0)   execution time : 4.009 s
Press any key to continue.
```

Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla

1. Análisis del problema

Entrada: Un número real X

Proceso: Verificar que el número cumpla las dos condiciones que son $x \geq 0$ y $x \leq 10$

Salida: Un mensaje indicando si el número pertenece o no al intervalo

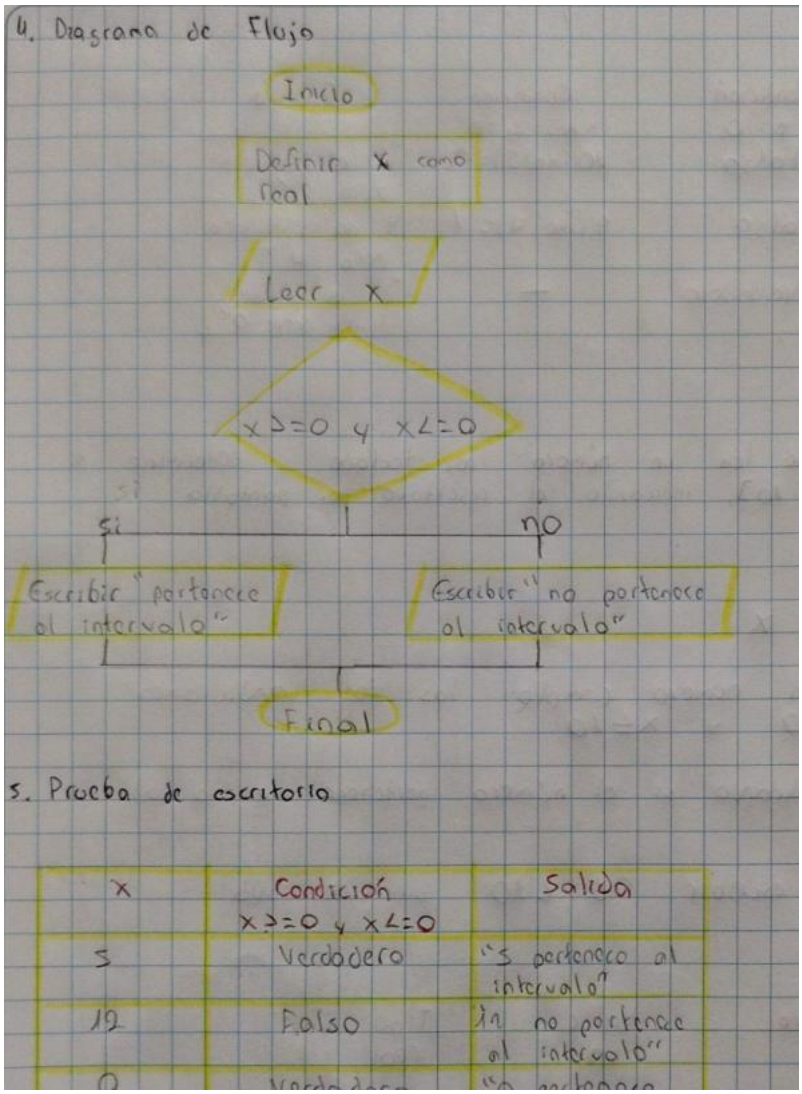
Restricción: No se debe escribir $0 \leq x \leq 10$ sino separado

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	X	Var	real
Límite inferior	0	cte	entero
Límite superior	10	cte	entero

3. Pseudocódigo

```
1.- Inicio
2.- Definir x como real
3.  Leer x
4   Si x>=0 Y x <= 10 entonces
5.-         Escribir x," pertenece al intervalo"
6.-     Sino
7.-         Escribir x," no pertenece al intervalo"
8.-     Fin Si
4.- Fin
```




```c
#include <stdio.h>

int main()
{
    float n1;

    printf("Ingrese numero 1: ");
    scanf("%f",&n1);

    if (n1>=0 && n1<=10)
        printf("El numero si esta en el intervalo\n");
        else
            printf("El numero no esta en el intervalo\n");



}
```

```
Ingrese numero 1: 6.3
El numero si esta en el intervalo

Process returned 0 (0x0)   execution time : 5.595 s
Press any key to continue.
```

```
Ingrese numero 1: 14
El numero no esta en el intervalo

Process returned 0 (0x0)   execution time : 2.200 s
Press any key to continue.
```

Problema 1.5 Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero C, interpretándolo como el número de segundos que dura un evento y calcule y muestre por pantalla cuatos días, horas, minutos y segundos representa

1. Análisis del problema

Entradas: Un valor entero C

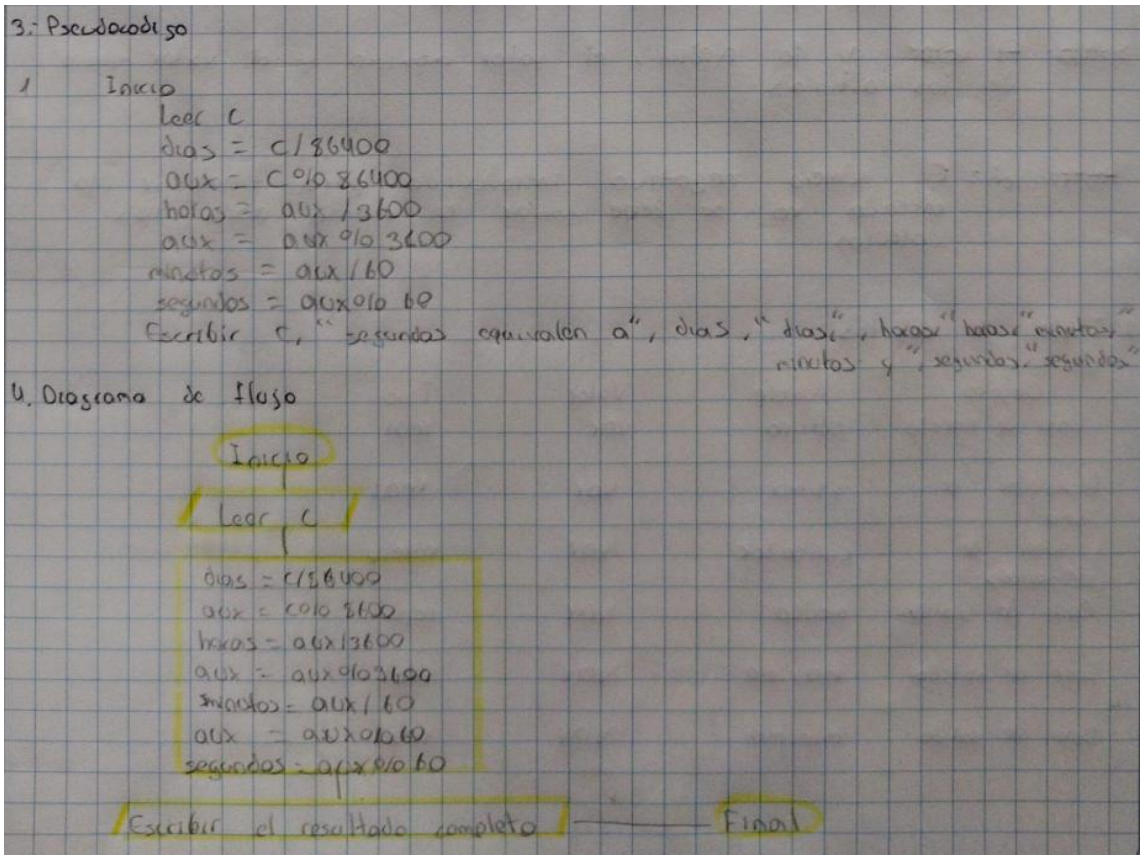
Proceso: Se utiliza la división entera y el operador modulo para descomponer los segundos en días, horas, minutos y segundos restantes
- Días: C/(24·60·60)
- Segundos: C%(24·60·60)
- Horas: (60·60)
- segundos al descontar las horas: (60·60)
- Minutos: C/60
- Segundos: modulo entre 60

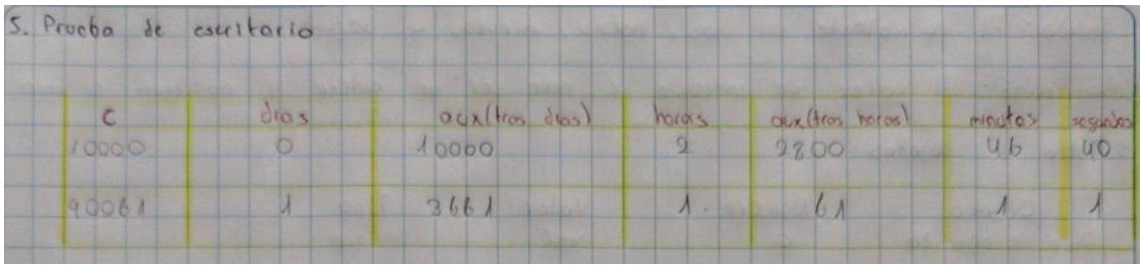
Salida: El equivalente en días, horas, minutos y segundos

Restricción: El valor de entrada C debe ser un entero y positivo o cero

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	C	var	entero
Dato de salida	dias	var	entero
Dato de salida	horas	var	entero
Dato de salida	minutos	var	entero
Dato de salida	segundos	var	entero
Variables auxiliares	aux	var	entero
Segundos en un día	86400	cte	entero
Segundos en una hora	3600	cte	entero
Segundos en un minuto	60	cte	entero






```c
#include <stdio.h>

int main()
{
    int n1, dias, minutos, segundos, horas, aux;

    printf("Ingrese numero 1: ");
    scanf("%d",&n1);
    dias=n1/86400;
    aux=n1%86400;
    horas=aux/3600;
    aux=aux%3600;
    minutos=aux/60;
    segundos=aux%60;
    printf("%d segundos equivalen a %d dias, %dhoras, %d minutos y %d segundos\n",n1,dias,horas,minutos,segundos);


}
```

```
Ingrese numero 1: 123456
123456 segundos equivalen a 1 dias, 10horas, 17 minutos y 36 segundos

Process returned 0 (0x0)   execution time : 5.085 s
Press any key to continue.
```

Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas

Análisis del problema

Entradas: Un número real indeterminado de alturas

Proceso: Se leen valores en un ciclo. Mientras la altura ingresada sea mayor o igual a 0, se acumula la suma de las alturas, se cuentan cuantas alturas válidas se ingresaron y se actualizan los valores máximo y mínimo
Al finalizar el ciclo, se calcula la media dividiendo la suma total entre el contador de alturas

Salida: El valor de la media, el valor máximo y el valor mínimo de las alturas

Restricción: El número negativo introducido para finalizar la entrada no se debe tomar en cuenta para los cálculos

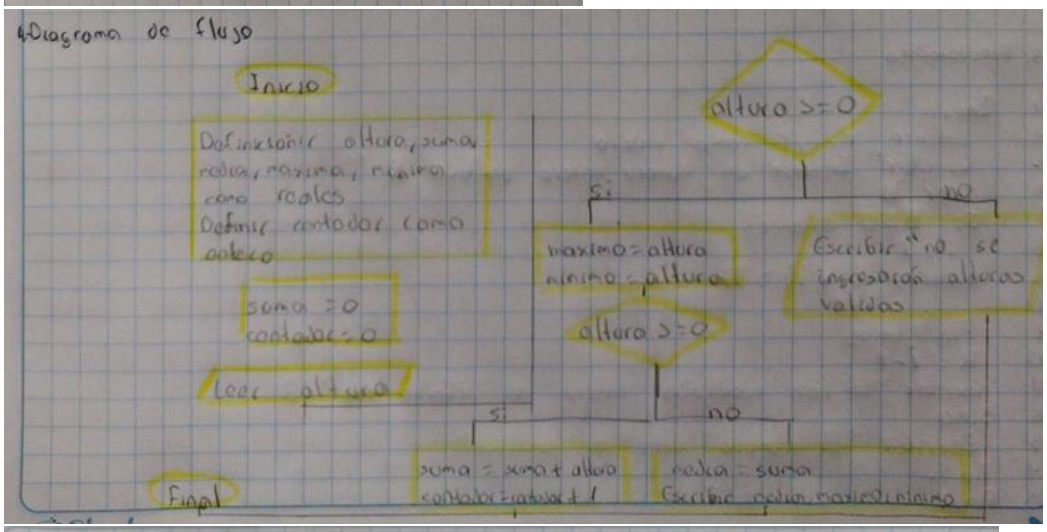
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	altura	var	real
Acumulador de suma	suma	var	real
Contador de elementos	contador	var	entero
Dato de salida	media	var	real
Dato de salida	maximo	var	real
Dato de salida	minimo	var	real

3. Pseudocodigo

```
1. Inicio
2.  Definir altura, suma, media, maximo, minimo como reales
3.  Definir contador como entero
4.  suma = 0
5.  contador = 0
6.  Escribir "Ingrese la altura"
7.  Leer altura
8.    Si altura >=0 entonces
9.        maximo = altura
10.       minimo = altura
11.   mientras altura >=0 hacer
12.       suma = suma + altura
13.       contador = contador + 1
14.   // Actualizar máximo y mínimo
15.   Si altura > maximo entonces
16.       maximo = altura
17.    FinSi
18.   Si altura < minimo entonces
19.       minimo = altura
20.    FinSi
21.    Escribir "Ingrese la altura" (negativo)
22.    Leer altura
23.   Fin Mientras
24.   Si contador >0 entonces
25.    media = suma / contador
26.    Escribir "La media de las alturas es:", media
27.    Escribir "La altura maxima es:", maximo
28.    Escribir "La altura minima es:", minima
29.   FinSi
30. Sino
31.    Escribir "No se ingresaron alturas validas"
32.  FinSi
33. Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

Altura	suma	contador	maximo	minimo	media	Salida
1.30; 1.40; 1.50; -1	5.10	3	1.70	1.60	1.70	la media es: 1.70 la altura con 1.50
-0.2	0	0	-	-	-	No se ingresaron alturas vali

```c
#include <stdio.h>

int main()
{
    float altura, suma = 0, media, maximo, minimo;
    int contador = 0;

    printf("Ingrese la altura: ");
    scanf("%f", &altura);

    if (altura >= 0) {
        maximo = altura;
        minimo = altura;
    }

    while (altura >= 0) {
        suma = suma + altura;
        contador = contador + 1;

        if (altura > maximo) {
            maximo = altura;
        }
        if (altura < minimo) {
            minimo = altura;
        }

        printf("Ingrese la altura: ");
        scanf("%f", &altura);
    }

    if (contador > 0) {
        media = suma / contador;
        printf("La media de las alturas es: %.2f\n", media);
        printf("La altura maxima es: %.2f\n", maximo);
        printf("La altura minima es: %.2f\n", minimo);
    } else
        printf("No se ingresaron alturas validas\n");


}
```

```
Ingrese la altura: 1.30
Ingrese la altura: 1.79
Ingrese la altura: 2
Ingrese la altura: 9
Ingrese la altura: -3
La media de las alturas es: 3.52
La altura maxima es: 9.00
La altura minima es: 1.30

Process returned 0 (0x0)   execution time : 29.306 s
Press any key to continue.
```

Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R \geq 0$ tal que $R^2 \leq N$

1. Análisis del problema

Entradas: Un número entero positivo N

Proceso: Se busca el mayor número entero $R \geq 0$ tal que $R^2 \leq N$. Siguiendo la pista de la guía, se utiliza una variable entera $i = 1$ que se incrementa de 1 en 1 mientras $i^2 \leq N$; la aproximación por defecto de la raíz es $R = i - 1$

Salida: El valor entero resultante de la raíz cuadrada por defecto

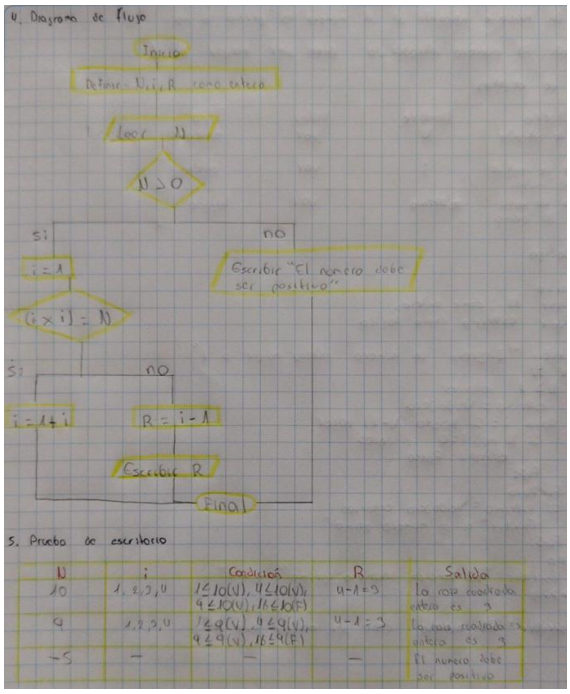
Restricción: El número de entrada N debe ser positivo

2: Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	N	var	entero
Variable de iteración	i	var	entero
Dato de salida	R	var	entero

3: Pseudocódigo

```
1   Inicio
2     Definir N, i, R como entero
3     Escribir "Ingrese un numero entero positivo N: "
4     Leer N
5     Si N >= 0 entonces
6       i = 1
7       mientras (i x i) <= N
8         i = i + 1
9       Fin Mientras
10      R = i - 1
11      Escribir "La raiz cuadrada entera por defecto es: ", R
12    Sino
13      Escribir "El numero debe ser positivo"
14    Fin Si
15  Final
```




```c
#include <stdio.h>

int main()
{
          int N, i, R;

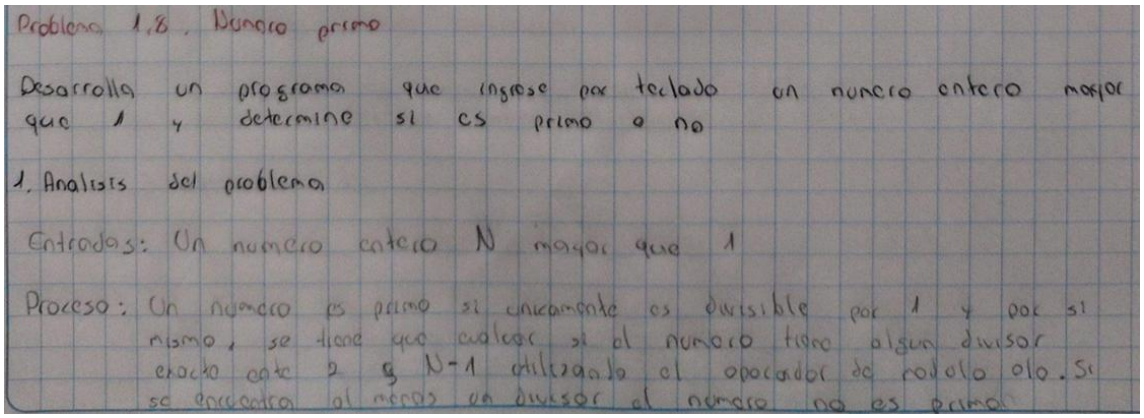
    printf("Ingrese un numero entero positivo N: ");
    scanf("%d", &N);

    if (N >= 0) {
        i = 1;
        while ((i * i) <= N) {
            i = i + 1;
        }
        R = i - 1;
        printf("La raiz cuadrada entera por defecto es: %d\n", R);
    } else
        printf("El numero debe ser positivo\n");


}
```

```
Ingrese un numero entero positivo N: 4
La raiz cuadrada entera por defecto es: 2

Process returned 0 (0x0)   execution time : 43.001 s
Press any key to continue.
```



Salida: Un mensaje indicando si el numero es primo o no

Restricción: El numero introducido debe ser mayor a 1

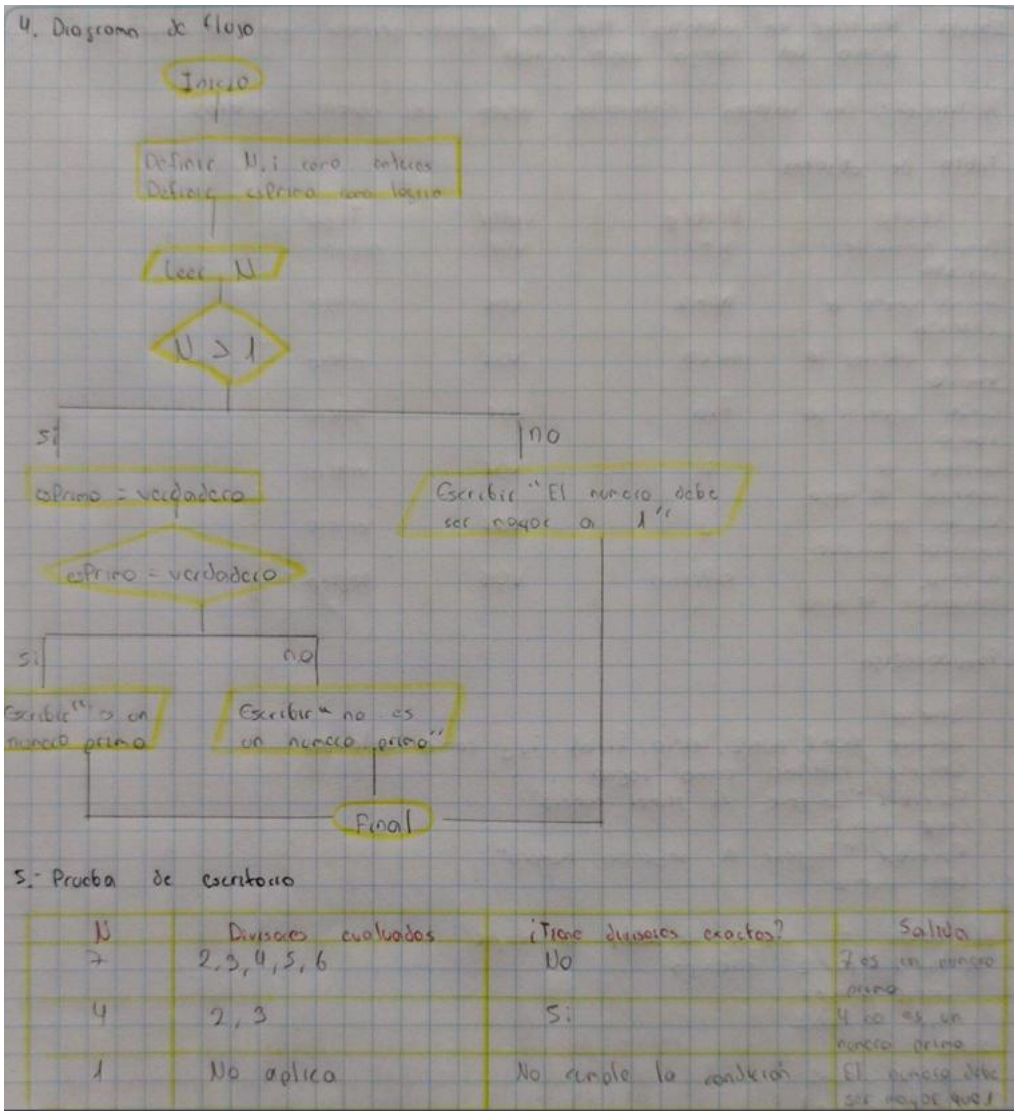
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	N	var	entero
Variable de control para el ciclo	i	var	entero
Bandera de validación de primo	esPrimo	var	logico

3. Pseudocodigo

```
Inicio
Definir N, i como enteros
Definir esPrimo como logico

Escribir "Ingrese un numero entero mayor a 1"
leer N
Si N > 1 entonces
    esPrimo = Verdadero
    Para i = 2 hasta (N-1) hacer
      Si N mod i = 0 entonces
          esPrimo = Falso
      FinSi
    FinPara
    Si esPrimo = Verdadero entonces
      Escribir "es un numero primo"
    FinSi
  sino
    Escribir "no es un numero primo"
  FinSi
Sino
  Escribir "El numero debe ser mayor que 1"
FinSi
Final
```



5.- Prueba de escritorio

N	Divisores evaluados	¿Tiene divisores exactos?	Salida
7	2, 3, 4, 5, 6	No	7 es un numero primo
4	2, 3	Si	4 no es un numero primo
1	No aplica	No cumple la condición	El numero debe ser mayor que 1

```c
#include <stdio.h>
#include <stdbool.h>
int main()
{
        int N, i;
    bool esPrimo;

    printf("Ingrese un numero entero mayor a 1: ");
    scanf("%d", &N);

    if (N > 1) {
        esPrimo = true;

        for (i = 2; i < N; i++) {
            if (N % i == 0) {
                esPrimo = false;
            }
        }

        if (esPrimo) {
            printf("Es un numero primo\n");
        } else {
            printf("No es un numero primo\n");
        }
    } else
        printf("El numero debe ser mayor que 1\n");

}
```

```
Ingrese un numero entero mayor a 1: 2
Es un numero primo

Process returned 0 (0x0)   execution time : 2.039 s
Press any key to continue.
```

```
Ingrese un numero entero mayor a 1: 4
No es un numero primo

Process returned 0 (0x0)   execution time : 4.412 s
Press any key to continue.
```

Problema 1.9. Numeros primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los numeros primos comprendidos entre dos numeros enteros introducidos por teclado

1. Analisis del problema

Entradas: Dos numeros enteros num1 y num2

Proceso: Se determina cual de los dos numeros es el menor y cual es el mayor para establecer el rango correcto
Se recorren todos los números enteros comprendidos en ese intervalo
Para cada numero del recorrido, se evalua si es un numero primo

Salida: Mostrar en pantalla todos los numeros primos que se encuentren dentro del rango especificado

Restricción: Los valores ingresados deben ser números enteros

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Primer número de entrada	num1	var	entero
Segundo numero de entrada	num2	var	entero
Limite inferior del rango	inicio	var	entero
Limite superior del rango	fin	var	entero
Variable de recorrido del rango	i	var	entero
Variable para comprobar divisores	j	var	entero
Bandera de validación de primo	esPrimo	var	logico

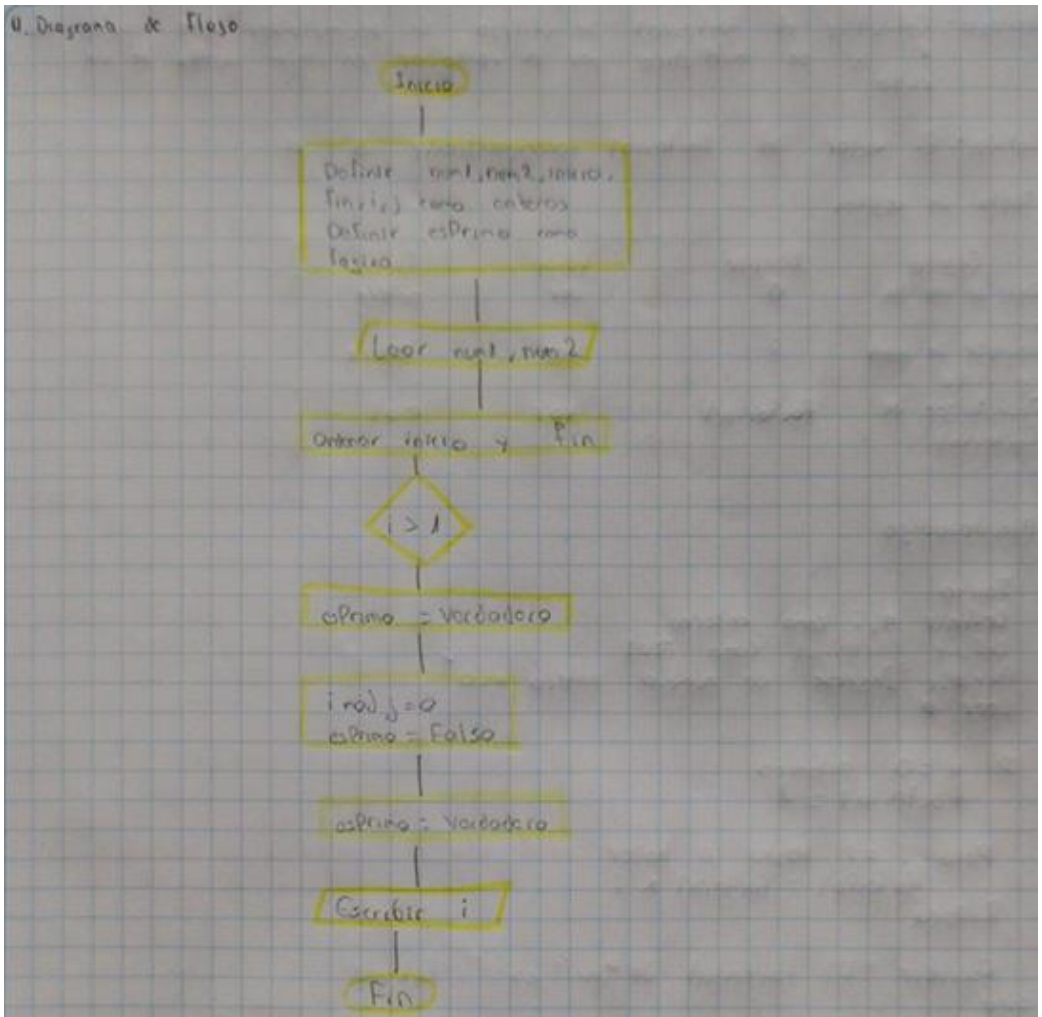
3. Pseudocodigo

```
Inicio
Definir num1, num2, inicio, final, i, j como enteros
Definir esPrimo como logico
Escribir "Ingrese el primer numero"
Leer num1
Escribir "Ingrese el segundo numero"
Leer num2

//Ordenar el rango de menor a mayor
Si num1 < num2 entonces
   inicio = num1
   fin = num2
sino
   inicio = num2
   fin = num1
FinSi
Escribir "Numeros primos entre", inicio "y" fin

Para i = inicio hasta fin hacer
   Si i > 1 entonces
      esPrimo = Verdadero
      Para j = 2 hasta (i-1) hacer
         Si Mod j = 0 entonces
            esPrimo = falso
         FinSi
      FinPara
      Si esPrimo = verdadero entonces
         Escribir i
      FinSi
Final
```

4. Diagrama de Flujo



5. Prueba de escritorio

Entradas (num1, num2)	Rango ordenado (inicio a Fin)	Números primos	Salida
10, 20	10 a 20	11, 13, 17, 19	Números primos: 11, 13, 17, 19
1, 10	1 a 10	2, 3, 5, 7	Números primos: 2, 3, 5, 7

```c
#include <stdio.h>

int main()
{
    int num1, num2, inicio, fin;
    int i, j, esPrimo;
    int encontrado = 0;
    printf("Ingrese el primer numero entero: ");
    scanf("%d", &num1);
    printf("Ingrese el segundo numero entero: ");
    scanf("%d", &num2);
    if (num1 > num2) {
        inicio = num2;
        fin = num1;
    } else {
        inicio = num1;
        fin = num2;
    }
    printf("\nNumeros primos comprendidos entre %d y %d:\n", inicio, fin);
    for (i = inicio; i <= fin; i++) {
        if (i <= 1) {
            continue;
        }
        esPrimo = 1;

        for (j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                esPrimo = 0;
                break;
            }
        }
        if (esPrimo == 1) {
            printf("%d ", i);
            encontrado = 1;
        }
    }
    if (!encontrado) {
        printf("No se encontraron numeros primos en este rango.");
    }
}
```

```
Ingrese el primer numero entero: 10
Ingrese el segundo numero entero: 30

Numeros primos comprendidos entre 10 y 30:
11 13 17 19 23 29
Process returned 0 (0x0)   execution time : 7.661 s
Press any key to continue.
```

Problema 1.10. Factorial

Desarrolla un programa que lea un numero entero $n \geq 0$ y calcule $n! = 1 \cdot 2 \cdot \ldots \cdot n$, sabiendo que $0! = 1$

1. Analisis del problema

Entradas: Un numero entero $n \geq 0$

Proceso: Calcular el factorial. Se utiliza un acumulador que inicia en 1 y se multiplica por el contador en cada vuelta de un ciclo

Salida: El valor del factorial correspondiente a n

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Variable de control del ciclo	i	var	entero
Acumulador de factorial	factorial	var	real

3. Pseudocodigo

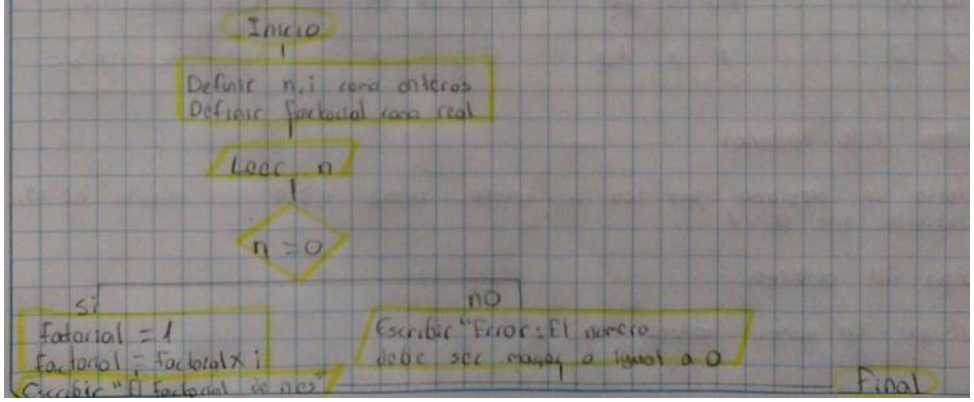
```
Inicio
Definir n, i como enteros
Definir factorial como real
Escribir "Ingrese un numero entero n>=0"
Leer n

Si n >=0 entonces
   factorial = 1

   Para i = 1 hasta n hacer
      factorial = factorial * i
   FinPara

   Escribir "El factorial de "n" es:", factorial
sino
   Escribir "Error: el número debe ser mayor o igual a 0"
FinSi
Fin
```

4. Diagrama de flujo



```c
#include <stdio.h>

int main()
{
        int n, i;
    double factorial = 1;

    printf("Ingrese un numero entero n >= 0: ");
    scanf("%d", &n);

    if (n >= 0) {
        factorial = 1;
        for (i = 1; i <= n; i++) {
            factorial = factorial * i;
        }
        printf("El factorial de %d es: %.0f\n", n, factorial);
    } else
        printf("Error: el numero debe ser mayor o igual a 0\n");

}
```

```
Ingrese un numero entero n >= 0: 3
El factorial de 3 es: 6

Process returned 0 (0x0)   execution time : 4.111 s
Press any key to continue.
```

Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero $n \geq 1$ y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...), donde cada termino es la suma de los dos anteriores

1. Análisis del problema

Entradas: Un numero entero $n \geq 1$

Proceso: Mostrar los primeros n términos de la serie de Fibonacci donde cada término es la suma de los dos anteriores. Se emplean tres variables: el término anterior, el termino actual y el termino siguiente

Salida: La secuencia ordenada con los primeros n términos de la serie

¿Como cambia tu algoritmo si solo debes mostrar el termino número n y no toda la serie?

No se debería imprimir cada valor en una secuencia ni acumularlos en pantalla. El ciclo simplemente se iteraría hasta la posición n actualizando los valores de los dos numeros anteriores para quedarse con el valor final buscado

2. Tabla de objetos

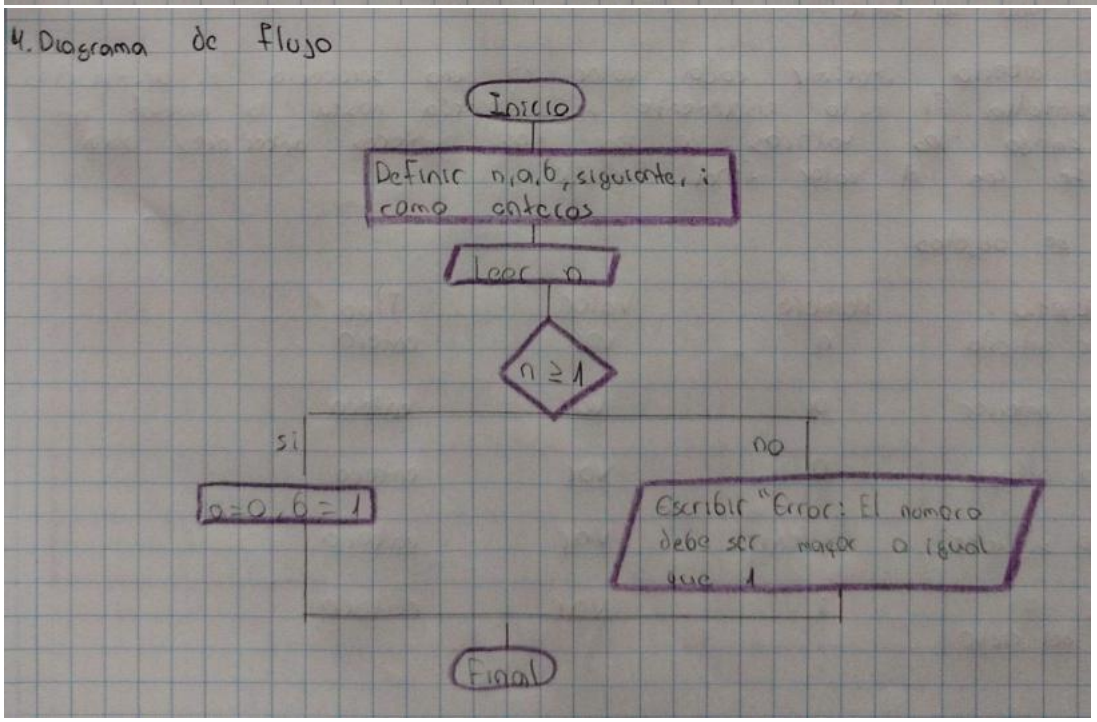
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Termino anterior	a	var	entero
Termino actual	b	var	entero
Termino siguiente	siguiente	var	entero
Variable de control del ciclo	i	var	entero

3: Pseudocodigo

```
Inicio
  Definir n,a,b, siguiente, i como entero
  Escribir "Ingrese un número entero n>=1"
  Leer n

  Si n >= 1 entonces
     a=0
     b=1
    Escribir "serie de fibonacci:"
    Para i=1 hasta n hacer
        Si i=1 entonces
           Escribir a
        Sino
           Si i=2 entonces
              Escribir b
           Sino
              siguiente = a+b
              a=b
              b= siguiente
              Escribir siguiente
           FinSi;
        FinSi
    FinPara
  Sino
     Escribir "Error: El numero debe ser mayor o menor que 1"
  FinSi;
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

n	Desarrollo de las iteraciones	Valores internos de a,b, siguiente	Salida
3	i:1->imprime 0 i:2->imprime 1 i:3->siguiente=1, b=1->imprime 1	Actualización progresiva de variables de soporte	0,1,1,
1	i:1->imprime 0	Valores iniciales	0
0	No aplica	No cumple condición (n>=1)	Error: El numero debe ser mayor o igual a 1

```c
#include <stdio.h>
int main()
{
        int n, a, b, siguiente, i;

    printf("Ingrese un numero entero n >= 1: ");
    scanf("%d", &n);

    if (n >= 1) {
        a = 0;
        b = 1;
        printf("Serie de Fibonacci:\n");

        for (i = 1; i <= n; i++) {
            if (i == 1) {
                printf("%d ", a);
            } else if (i == 2) {
                printf("%d ", b);
            } else {
                siguiente = a + b;
                a = b;
                b = siguiente;
                printf("%d ", siguiente);
            }
        }
        printf("\n");
    } else {
        printf("Error: El numero debe ser mayor o igual que 1\n");
    }
}
```

```
Ingrese un numero entero n >= 1: 10
Serie de Fibonacci:
0 1 1 2 3 5 8 13 21 34

Process returned 0 (0x0)   execution time : 2.280 s
Press any key to continue.
```

Problema 1.12 Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos

1. Análisis del problema

Entradas: Un número entero positivo n

Proceso: Se utiliza el operador módulo para extraer el último dígito del número, se acumula dicho valor en una variable de suma y se reduce el número mediante una división entera entre 10 para eliminar el dígito procesado

Salida: El valor total de la suma de todos los dígitos que componen el número

Restricción: El número de entrada no puede ser negativo ni cero

¿Cuantas veces se repite tu ciclo para un número de K dígitos?

El ciclo se repite exactamente K veces, ya que en cada iteración la división entera entre 10 elimina un dígito de derecha a izquierda

2. Tabla de objetos

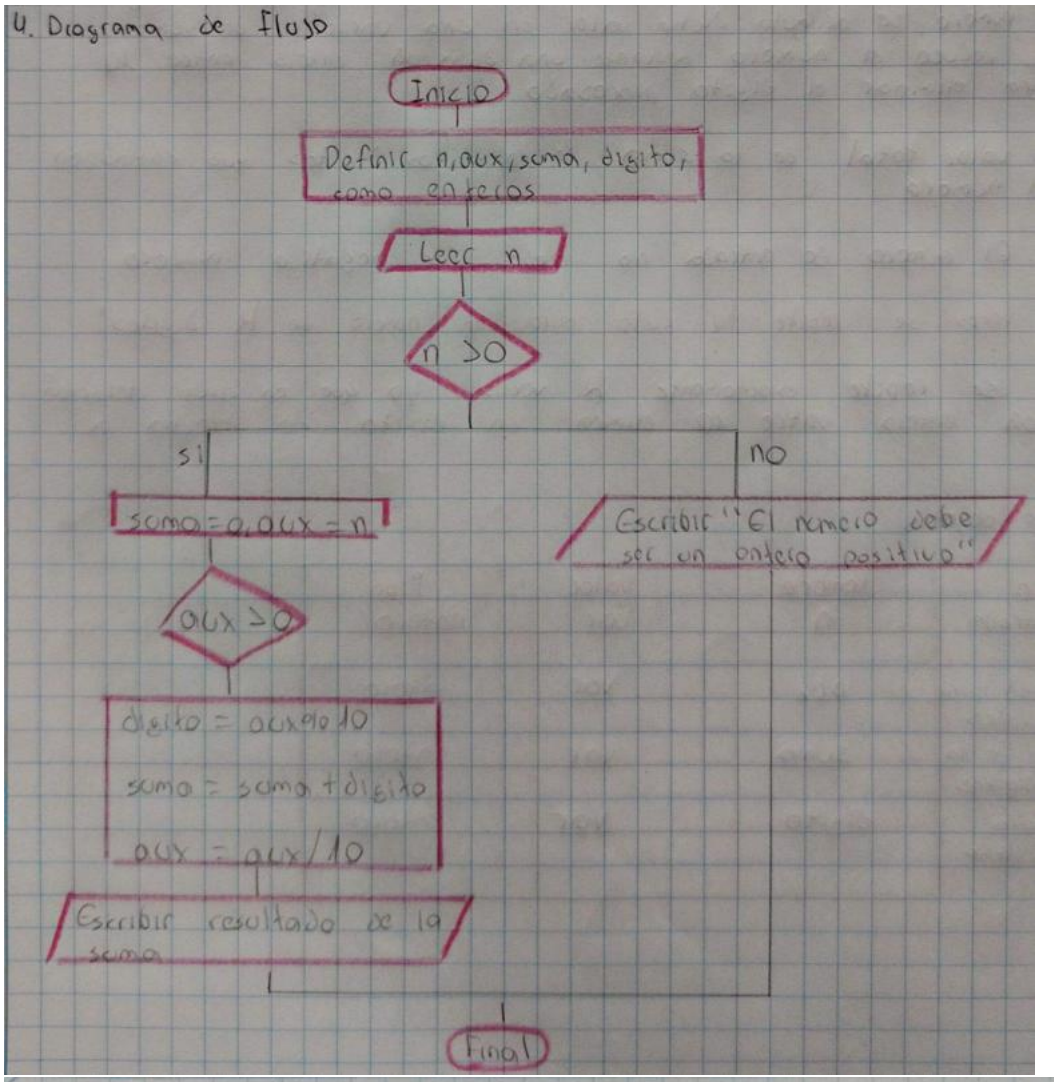
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Copia temporal para las operaciones	aux	var	entero
Acumulador de la suma de dígitos	suma	var	entero
Variable para extraer el último dígito	digito	Var	entero

3. Pseudocodigo

```
Inicio
  Definir n,aux, suma, digito como enteros

Escribir "Ingrese un número entero positivo"
Leer n
si n > 0 entonces
    suma = 0
    aux = n
    Mientras aux > 0 hacer
       digito = aux %% 10
       suma = suma + digito
       aux = aux / 10
    FinMientras
    Escribir "La suma de los digitos de", n, es:", su
  Sino
    Escribir "El numero debe ser entero positivo"
  FinSi
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

n	aux inicial	Iteraciones del bucle	suma acumulada
4825	4825	digito=5 aux=482 digito=2 aux=48 digito=8 aux=4 digito=4 aux=0	19
7	7	digito=7, aux=0	7
-12	No aplica		0

```c
#include <stdio.h>

int main()
{
        int n, aux, suma, digito;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

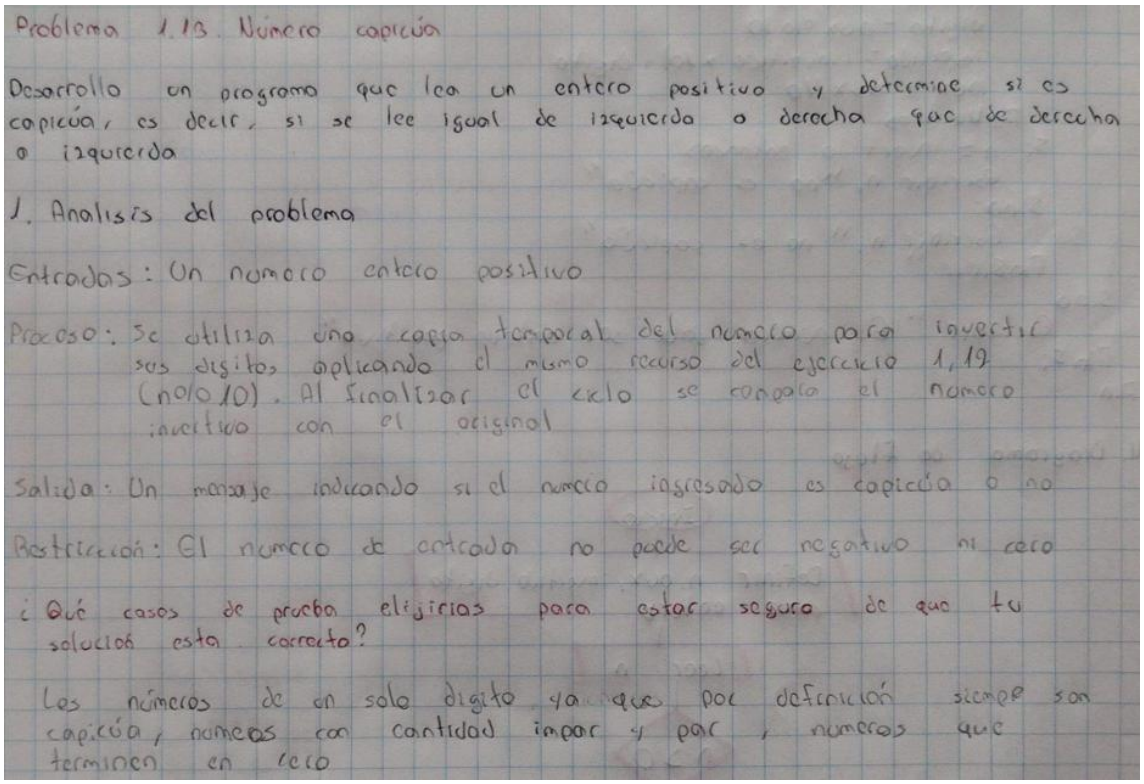
    if (n > 0) {
        suma = 0;
        aux = n;

        while (aux > 0) {
            digito = aux % 10;
            suma = suma + digito;
            aux = aux / 10;
        }

        printf("La suma de los digitos de %d es: %d\n", n, suma);
    } else {
        printf("El numero debe ser entero positivo\n");
    }
}
```

```
Ingrese un numero entero positivo: 123
La suma de los digitos de 123 es: 6

Process returned 0 (0x0)   execution time : 89.315 s
Press any key to continue.
```



Problema 1.13. Numero capicúa

Desarrollo un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda

1. Analisis del problema

Entradas: Un numero entero positivo

Proceso: Se utiliza una copia temporal del numero para invertir sus digitos aplicando el mismo recurso del ejercicio 1.12 (n%10). Al finalizar el ciclo se compara el numero invertido con el original

Salida: Un mensaje indicando si el numero ingresado es capicúa o no

Restricción: El numero de entrada no puede ser negativo ni cero

¿Qué casos de prueba elijirias para estar seguro de que tu solución esta correcta?

Los números de un solo digito ya que por definición siempre son capicúa, numeros con cantidad impar y par, numeros que terminen en cero

2. Tabla de objetos

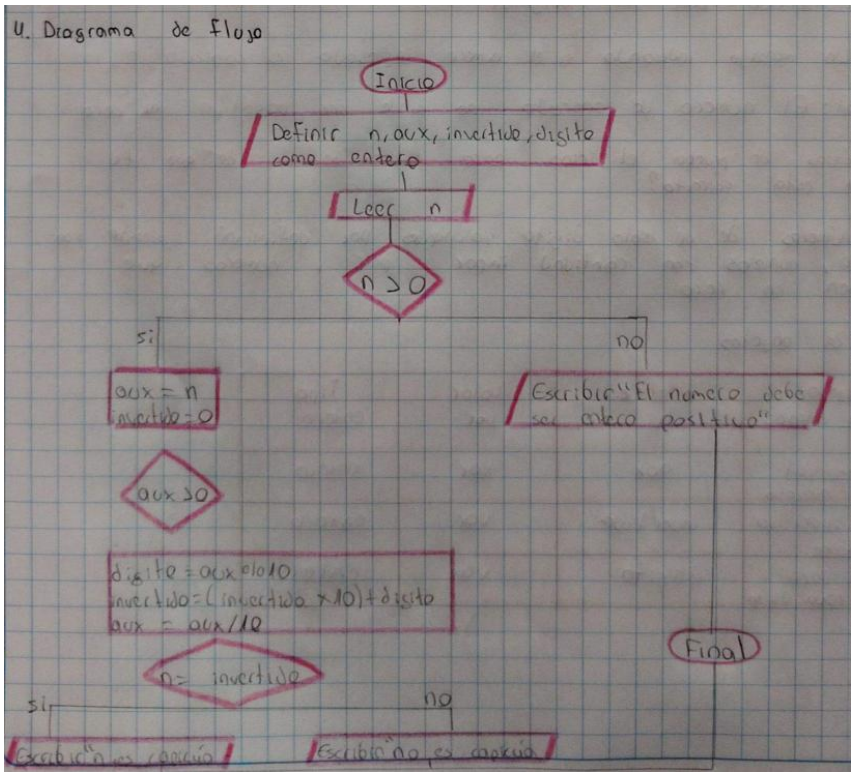
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Copia temporal para el proceso	aux	var	entero
Número invertido resultante	invertido	var	entero
Variable para extraer cada dígito	digito	var	entero

3. Pseudocodigo

```
Inicio
Definir n, aux, invertido, digito como entero
Escribir "Ingrese un numero entero positivo"
Leer n

Si n >= 0 entonces
    aux = n
    invertido = 0
    Mientras aux > 0 hacer
        digito = aux % 10
        invertido = (invertido * 10) + digito
        aux = aux / 10
    FinMientras
    Si n = invertido entonces
        Escribir n, " es capicua"
    Sino
        Escribir n, " no es capicua"
    FinSi
Sino
    Escribir "El numero debe ser entero positivo"
FinSi
Fin
```

4. Diagrama de flujo




```
Inicio
Definir n, aux, invertido, digito como entero
Escribir "Ingrese un numero entero positivo"
Leer n

Si n > 0 entonces
    aux = n
    invertido = 0
    Mientras aux > 0 hacer
        digito = aux % 10
        invertido = (invertido * 10) + digito
        aux = aux / 10
    FinMientras
    Si n = invertido entonces
        Escribir n, "es capicua"
    Sino
        Escribir n, "no es capicua"
    FinSi
Sino
    Escribir "El numero debe ser entero positivo"
FinSi
Fin
```

```c
#include <stdio.h>

int main()
{
        int n, aux, invertido, digito;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

    if (n > 0) {
        aux = n;
        invertido = 0;

        while (aux > 0) {
            digito = aux % 10;
            invertido = (invertido * 10) + digito;
            aux = aux / 10;
        }

        if (n == invertido) {
            printf("%d es capicua\n", n);
        } else {
            printf("%d no es capicua\n", n);
        }
    } else
        printf("El numero debe ser entero positivo\n");
}
```

```
Ingrese un numero entero positivo: 12321
12321 es capicua

Process returned 0 (0x0)   execution time : 34.797 s
Press any key to continue.
```

```
Ingrese un numero entero positivo: 1234
1234 no es capicua

Process returned 0 (0x0)   execution time : 2.716 s
Press any key to continue.
```

Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a%b). Al terminar, a es el MCD

1. Análisis del problema

Entradas: Dos números enteros positivos a y b

Proceso: Se aplica el algoritmo de Euclides mediante un ciclo mientras b ≠ 0. Cada iteración se reemplaza utilizando una variable auxiliar. Al terminar el ciclo, el valor almacenado en a es el MCD

Salida: El valor correspondiente al MCD de los dos números ingresados

Restricción: Los valores de entrada no pueden ser negativos ni cero

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Primer número de entrada	a	var	entero
Segundo número de entrada	b	var	entero
Variable auxiliar para el intercambio	temp	var	entero

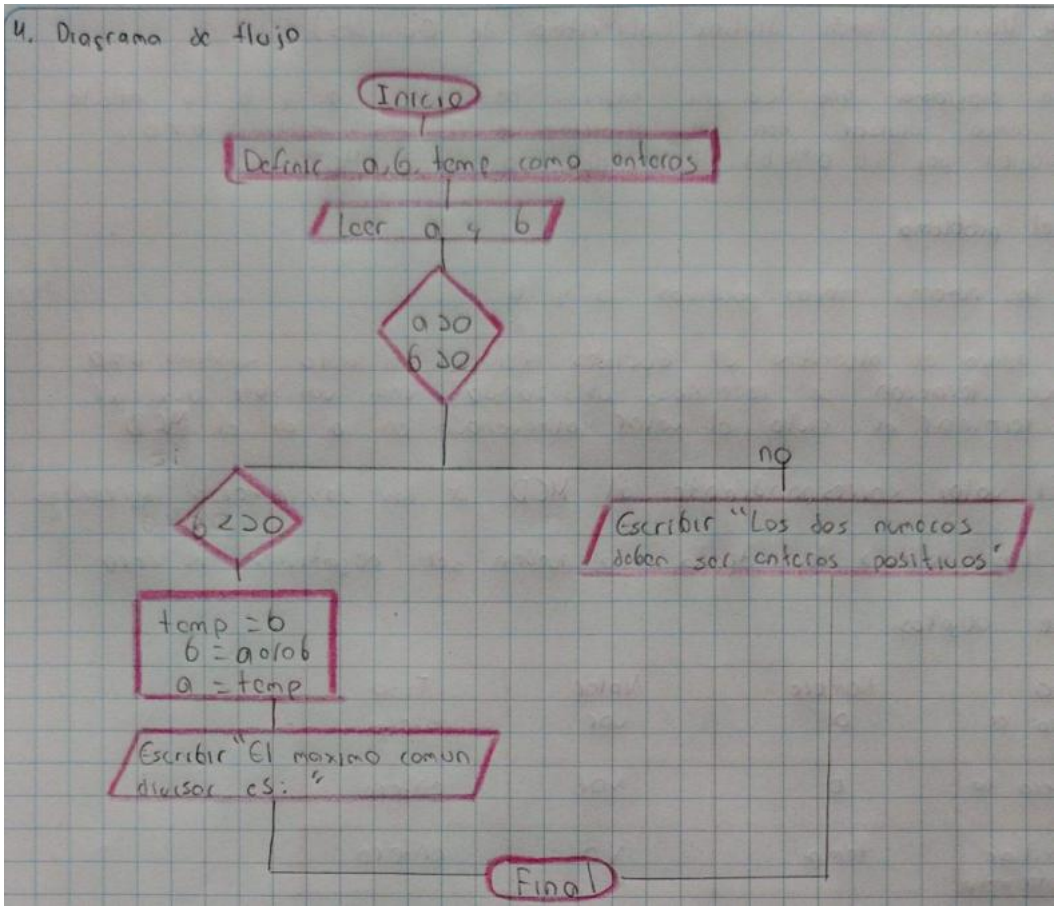
3. Pseudocódigo

```
Inicio
    Definir a, b, temp como entero
    Escribir "Ingrese el primer numero entero positivo"
    Leer a
    Escribir "Ingrese el segundo numero entero positivo"
    Leer b

    Si a>0 y b>0 entonces
        Mientras b <> 0 hacer
            temp = b
            b = a%b
            a = temp
        FinMientras

        Escribir "El máximo común divisor es: "
    sino
        Escribir "Ambos números deben ser enteros positivos"
    FinSi
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

Entradas iniciales a, b	Iteraciones (b <> 0)	Estado de las variables	Salida
48, 18	1, 18 70 Verdadero 2, 12 70 Verdadero 3, 6 70 Verdadero 4, 0 70 Falso	temp=18 b=18%18=18 a=18 temp=12 b=18%12=12 a=18 temp=6 b=12%6=0 a=12	El maximo comun divisor es 6
0, 5	No aplica	No cumple la condición	Ambos numeros deben ser enteros

```c
#include <stdio.h>

int main()
{
        int a, b, temp;

    printf("Ingrese el primer numero entero positivo: ");
    scanf("%d", &a);
    printf("Ingrese el segundo numero entero positivo: ");
    scanf("%d", &b);

    if (a > 0 && b > 0) {
        while (b > 0) {
            temp = b;
            b = a % b;
            a = temp;
        }
        printf("El maximo comun divisor es: %d\n", a);
    } else
        printf("Ambos numeros deben ser enteros positivos\n");
}
```

```
Ingrese el primer numero entero positivo: 48
Ingrese el segundo numero entero positivo: 18
El maximo comun divisor es: 6

Process returned 0 (0x0)   execution time : 33.641 s
Press any key to continue.
```

Problema 1,15 Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si tambien son divisibles entre 400

1. Analisis del problema

Entradas: Un numero entero que representa un año

Proceso: Determinar si el año es bisiesto evaluando si es divisible entre 4 con la salvedad de que los años divisibles entre 100 solo lo son si tambien son divisibles entre 400

Salida: Un mensaje indicando si el año ingresado es bisiesto o no

Restricción: El año no puede ser negativo ni cero

¿Es equivalente escribir las condiciones en otro orden?

Depende de cómo se estructuren los operadores lógicos. Si se evalua primero si es divisible entre 4 sin aislar la excepción de los siglos y el resultado por 1900 sería incorrecto

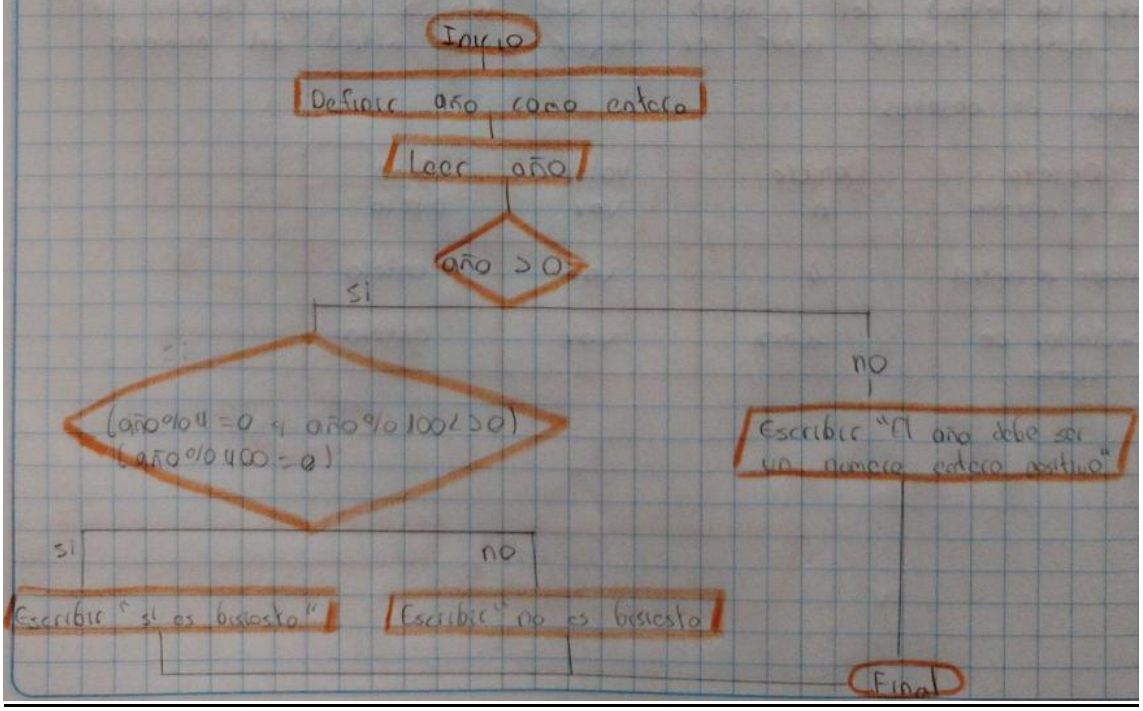
2.- Tabla de objetos

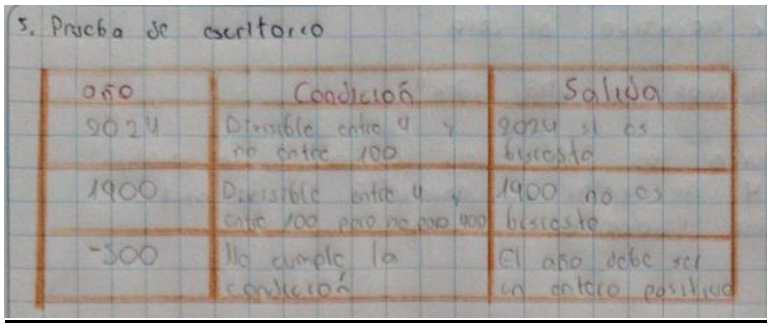
Objeto	Nombre	Valor	Tipo
Dato de entrada	año	var	entero

3.- Pseudocodigo

```
Inicio
  Definir año como entero
  Escribir "Ingrese un año"
  Leer año
  Si año >0 entonces
     Si (año%4=0 y año%100<>0) o (año%400=0) entonces
        Escribir "si es bisiesto"
     sino
        Escribir "no es bisiesto"
     FinSi
  sin
     Escribir "El año debe ser un numero entero positivo"
  FinSi
Fin
```

4. Diagrama de Flujo





5. Prueba de escritorio

año	Condición	Salida
2024	Divisible entre 4 y no entre 100	2024 sí es bisiesto
1900	Divisible entre 4 y entre 100 pero no 400	1900 no es bisiesto
-300	No cumple la condición	El año debe ser un entero positivo

```c
#include <stdio.h>

int main()
{
        int anio;

    printf("Ingrese el valor: ");
    scanf("%d", &anio);

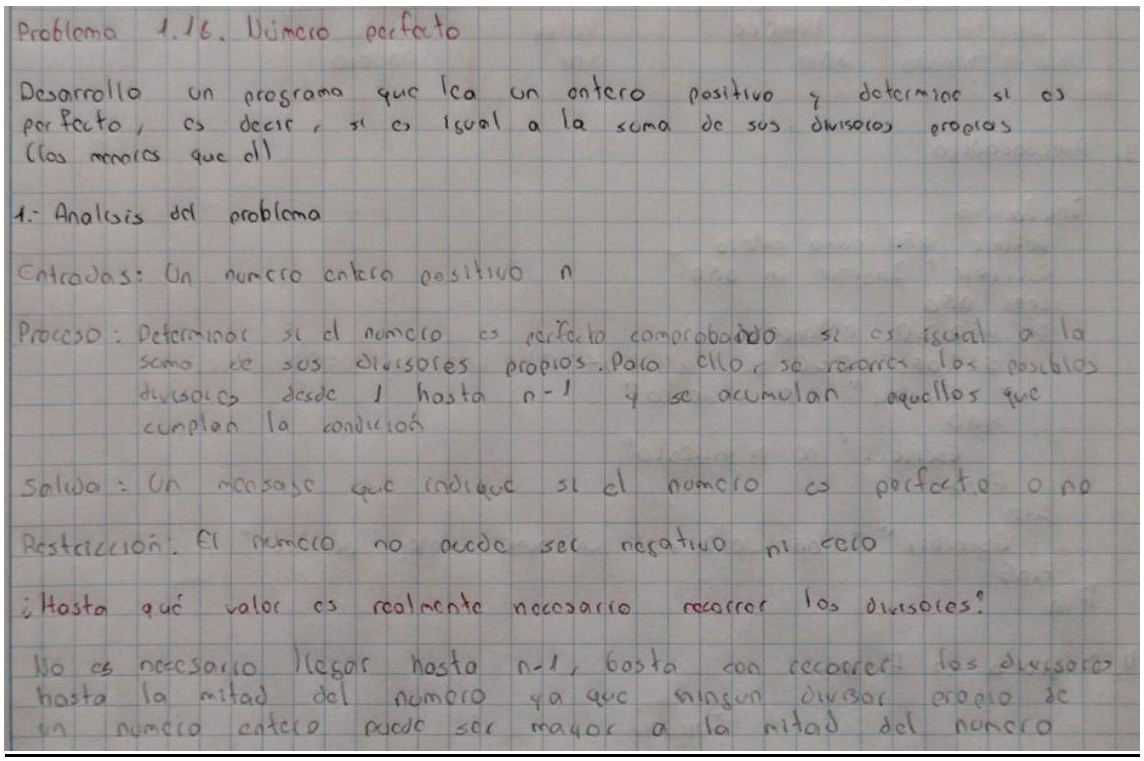
    if (anio > 0) {
        if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
            printf("si es bisiesto\n");
        } else {
            printf("no es bisiesto\n");
        }
    } else
        printf("El ano debe ser un numero entero positivo\n");
}
```

```
Ingrese el valor: 2024
si es bisiesto

Process returned 0 (0x0)   execution time : 9.598 s
Press any key to continue.
```

```
Ingrese el valor: 1900
no es bisiesto

Process returned 0 (0x0)   execution time : 12.618 s
Press any key to continue.
```



Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él)

1.- Análisis del problema

Entradas: Un número entero positivo n

Proceso: Determinar si el número es perfecto comprobando si es igual a la suma de sus divisores propios. Para ello, se recorren los posibles divisores desde 1 hasta n-1 y se acumulan aquellos que cumplen la condición

Salida: Un mensaje que indique si el número es perfecto o no

Restricción: El número no puede ser negativo ni cero

¿Hasta qué valor es realmente necesario recorrer los divisores?

No es necesario llegar hasta n-1, basta con recorrer los divisores hasta la mitad del número ya que ningún divisor propio de un número entero puede ser mayor a la mitad del número

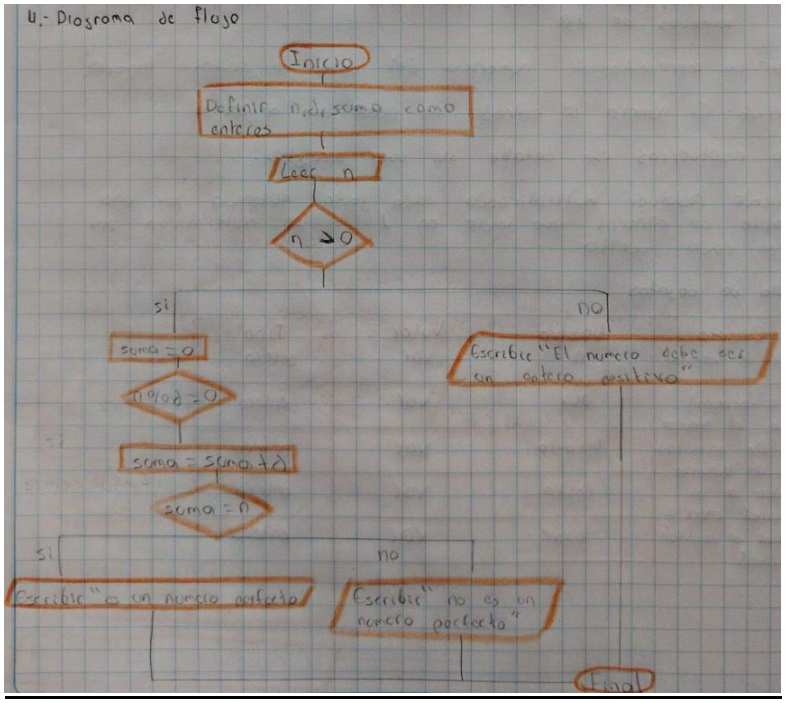
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Variable de control	d	var	entero
Acumulador de la suma	suma	var	entero

3. Pseudocodigo

```
Inicio
  Definir n,d,suma como enteros
  Escribir "Ingrese un numero entero positivo"
  Leer n
  Si n>0 entonces
      suma=0
      Para d=1 hasta (n-1) hacer
          Si n%d=0 entonces
              suma = suma+d
          FinSi
      FinPara
      si suma = n entonces
          Escribir n," es un número perfecto"
      Sino
          Escribir n,"no es un número perfecto"
      FinSi
  Sino
      Escribir "El numero debe ser entero positivo
  FinSi
Fin
```

4.- Diagrama de flujo



3.-Prueba de escritorio

n	Divisores propios	Suma acumulada	Comparacion	Salida
6	1, 2, 3	1+2+3 = 6	6 = 6 verdadero	6 es un numero perfecto
10	1, 2, 5	1+2+5 = 8	10 = 8 falso	10 no es un numero perfecto
-4	No aplica	No aplica	No cumple la condicion	El numero debe ser entero positivo

```c
#include <stdio.h>

int main()
{
        int n, d, suma;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &n);

    if (n > 0) {
        suma = 0;
        for (d = 1; d <= n - 1; d++) {
            if (n % d == 0) {
                suma = suma + d;
            }
        }

        if (suma == n) {
            printf("%d es un numero perfecto\n", n);
        } else {
            printf("%d no es un numero perfecto\n", n);
        }
    } else
        printf("El numero debe ser entero positivo\n");
}
```

```
Ingrese un numero entero positivo: 28
28 es un numero perfecto

Process returned 0 (0x0)   execution time : 25.164 s
Press any key to continue.
```

Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10, muestre su representación en binario

1. Análisis del problema

Entradas: Un numero entero positivo en base 10 n

Proceso: Se divide el número sucesivamente entre 2 y se obtienen los restos, como los restos se generan en orden inverso al que deben leerse, para resolver este detalle se puede ir construyendo el numero multiplicando cada resto por potencias sucesivas de 10

Salida: La representación numerica del valor en sistema binario

Restricción: El numero de entrada no puede ser negativo

¿Como resolverias ese detalle sin usar arreglos?

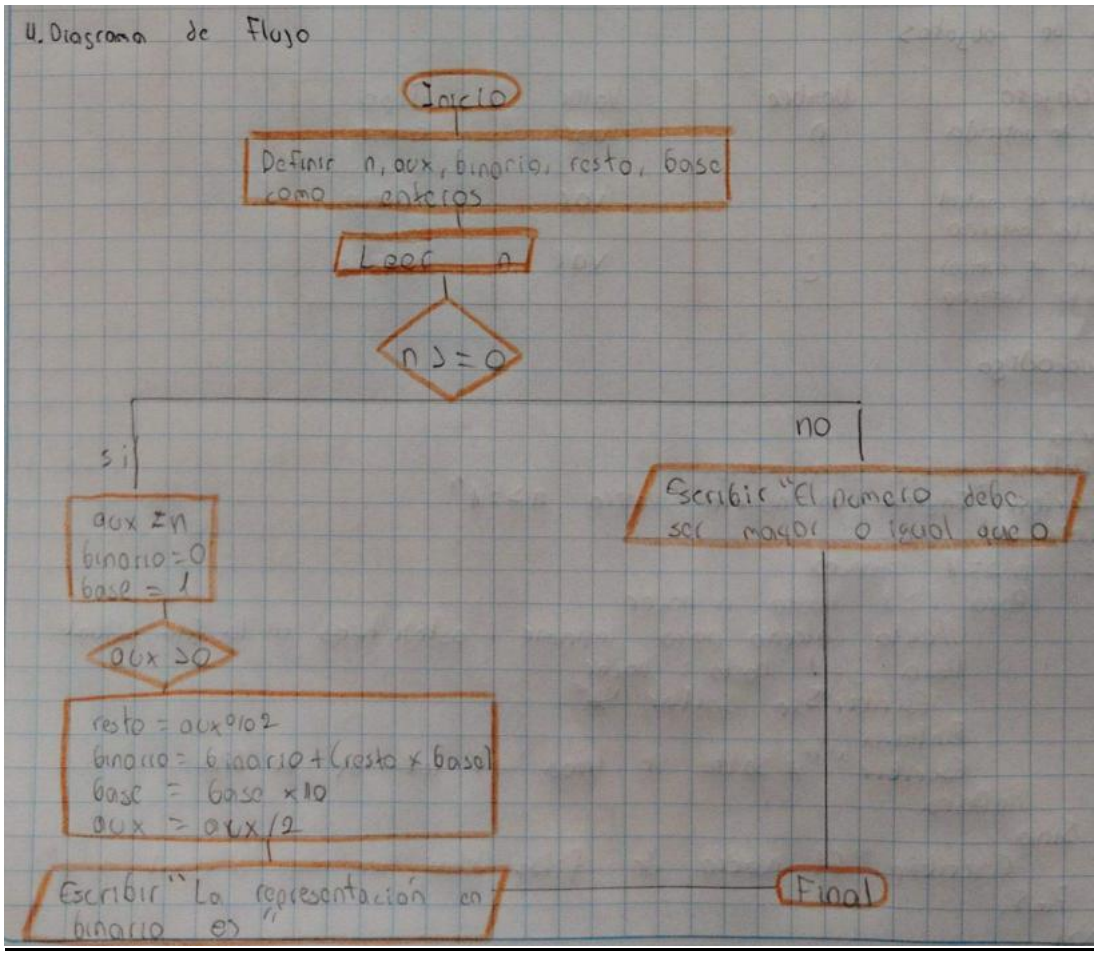
Se puede resolver multiplicando cada resto obtenido por una variable de control que inicia en 1 y se multiplican por 10 en cada vuelta

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Copia temporal para el proceso	aux	var	entero
Variable acumuladora del binario	binario	var	entero
Residuo de la división entre 2	resto	var	entero
Control de posición decimal	base	var	entero

3. Pseudocodigo

```
Inicio
  Definir n, aux, binario, resto, base como entero
  Escribir "Ingrese un entero positivo base 10"
  Leer n
  Si n >= 0 entonces
     aux = n
     binario = 0
     base = 1
     Si n = 0 entonces
        binario = 0
     Sino
        Mientras aux > 0 Hacer
           resto = aux % 2
           binario = binario + (resto * base)
           base = base * 10
           aux = aux / 2
        FinMientras
     FinSi
     Escribir "La representación binaria es: ", binario
  Sino
     Escribir "El número debe ser mayor o igual que 0"
  FinSi
Fin
```



5. Prueba de escritorio

n	aux inicial	Iteraciones	Salida
0	0	n>0	La representación en binario es 0
-2	no aplica	No cumple la condición	El numero debe ser mayor o igual que 0

```c
#include <stdio.h>

int main()
{
        int n, aux, binario, resto, base;

    printf("Ingrese un entero positivo base 10: ");
    scanf("%d", &n);

    if (n >= 0) {
        aux = n;
        binario = 0;
        base = 1;

        if (n == 0) {
            binario = 0;
        } else {
            while (aux > 0) {
                resto = aux % 2;
                binario = binario + (resto * base);
                base = base * 10;
                aux = aux / 2;
            }
        }

        printf("La representacion en binario es: %d\n", binario);
    } else
        printf("El numero debe ser mayor o igual que 0\n");
}
```

```
Ingrese un entero positivo base 10: 13
La representacion en binario es: 1101

Process returned 0 (0x0)   execution time : 19.807 s
Press any key to continue.
```

Problema 1.18. Triangulo de astedisticos

Desarrolla un programa que lea un entero $n \geq 1$ y dibuje por pantalla un triangulo rectangulo de n filas, donde la fila i tiene i astericticos

1. Analisis del problema

Entradas: Un numero entero $n \geq 1$

Proceso: Mediante un ciclo anidados: un ciclo externo que controla los filas y un ciclo interno que imprime los astedisticos correspondientes a cada fila

Salida: La representación grafica del triangulo rectangulo formado por astedisticos

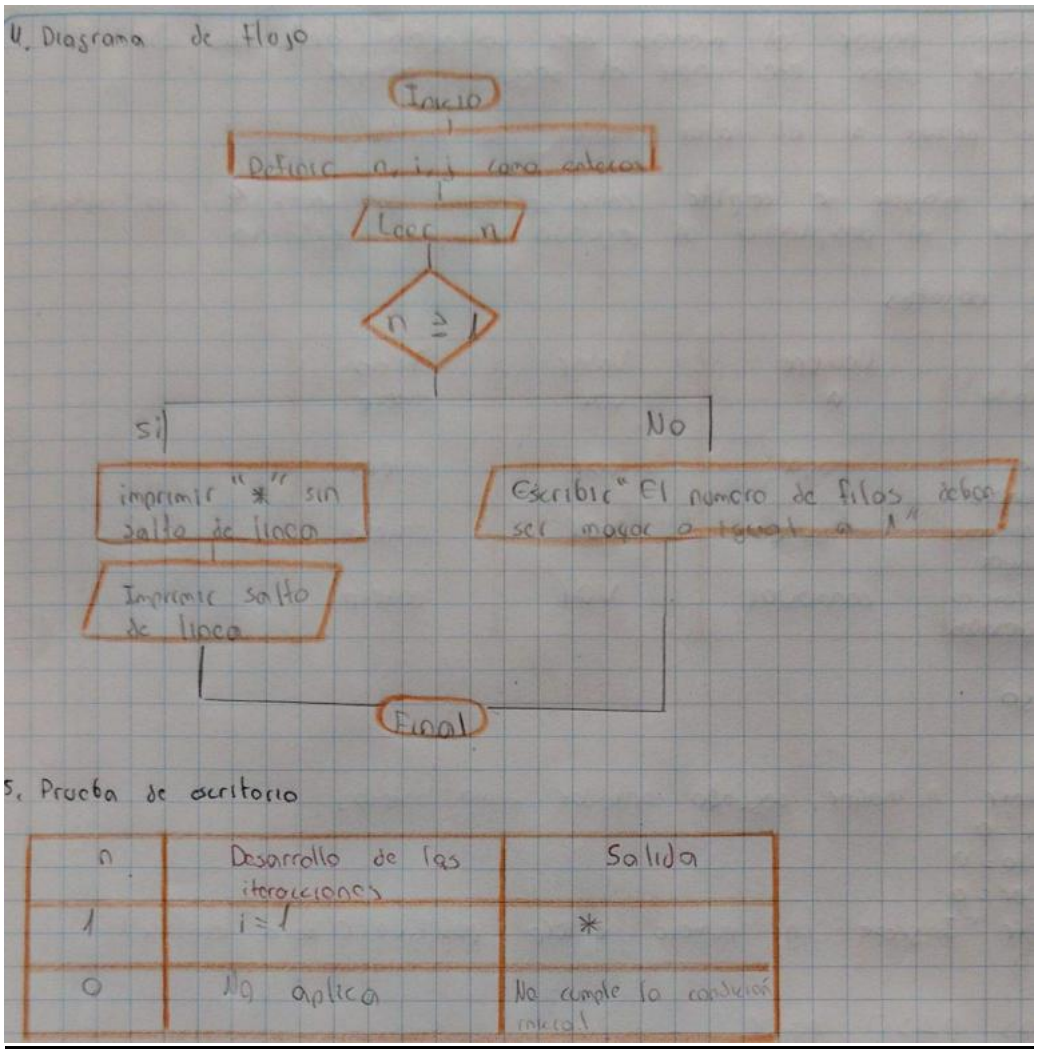
Restriccion: El numero debe ser mayor o igual que 1

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Variable de control del ciclo externo	i	var	entero
Variable de control del ciclo interno	j	var	entero

3. Pseudocodigo

```
Inicio
  Definir n,i,j como enteros
  Escribir "Ingrese un número entero n>=1"
  Leer n
  Si n>=1 entonces
     Para i=1 hasta n hacer
        //ciclo interno para imprimir i astedisticos en la fila actual
        Para j=1 hasta i hacer
           Escribir "*" sin saltar "*"
        FinPara
        Escribir "" // salto de linea al terminar de fila
     FinPara
  Sino
     Escribir "El numero de filas deben ser mayo o igual a 1"
  FinSi
Final
```




```c
#include <stdio.h>

int main()
{
        int n, i, j;

    printf("Ingrese un numero entero n >= 1: ");
    scanf("%d", &n);

    if (n >= 1) {
        for (i = 1; i <= n; i++) {

            for (j = 1; j <= i; j++) {
                printf("*");
            }
            printf("\n");
        }
    } else
        printf("El numero de filas deben ser mayor o igual a 1\n");
}
```

```
Ingrese un numero entero n >= 1: 4
*
**
***
****

Process returned 0 (0x0)   execution time : 16.625 s
Press any key to continue.
```

Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea numeros enteros positivos hasta que se introduzca un 0 y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo

1. Analisis del problema

Entradas: Una secuencia de numeros enteros positivos de forma sucesiva hasta que se introduzca el valor de 0

Proceso: Se leen los numeros en un ciclo mientras sean diferentes de cero, manteniendo un registro dinamico del valor mayor y del segundo valor mayor. Si llega un valor mayor que el actual maximo, el antiguo maximo pasa a ser el segundo mayor.

Salida: Mostrar el valor mayor y el segundo mayor de los numeros ingresados

Restriccion: Deben haber al menos dos valores distintos de 0 para poder determinar el segundo mayor

¿Qué deberia pasar si el valor mayor se repite?

Si el valor mayor se repite como ingresar dos veces 9, no altera ni al mayor ni desplaza al segundo mayor legitimo

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Variable que almacena el mayor	mayor	var	entero
Variable que almacena el segundo mayor	segundo	var	entero
Contador de numeros validos ingresados	contador	var	entero

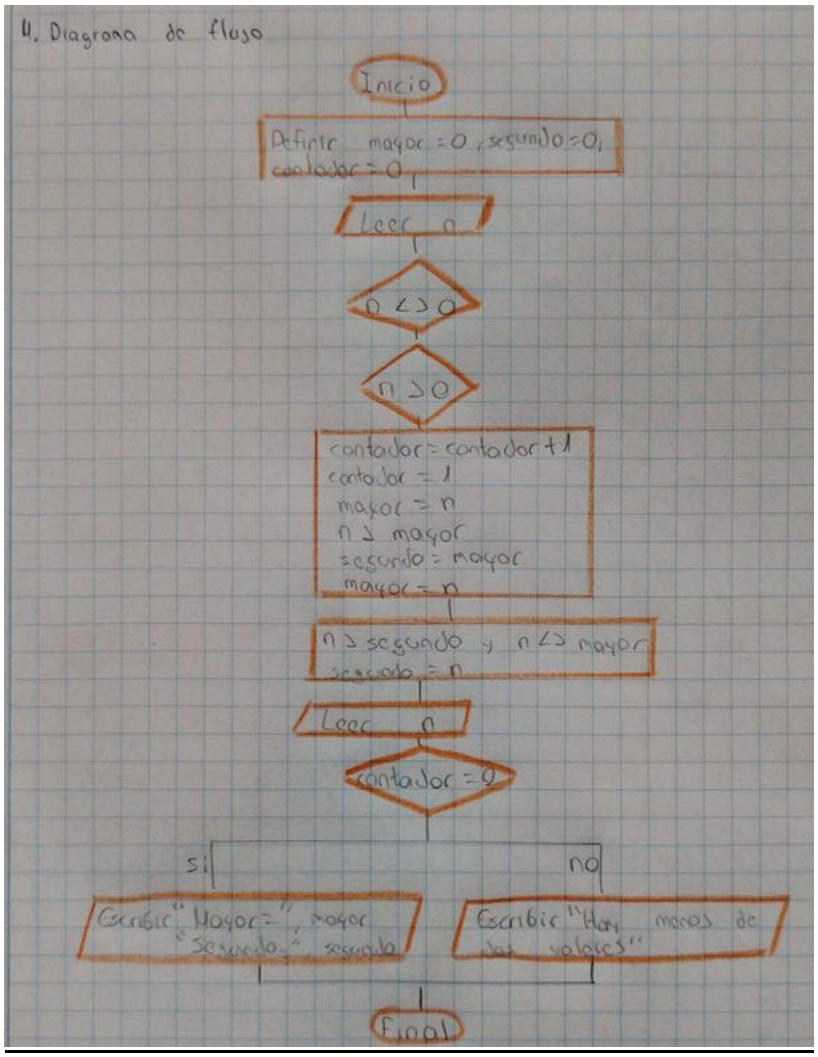
3. Pseudocodigo

```
Inicio
  Definir n, mayor, segundo, contador como enteros
  mayor = 0
  segundo = 0
  contador = 0
  Escribir "Ingrese un numero entero positivo (0 para terminar)"
  Leer n

  Mientras n <> 0 hacer
     Si n>0 entonces
        contador = contador+1

        si contador = 1 entonces
           mayor = n
        Sino
           Si n > mayor entonces
              segundo = mayor
              mayor = n
           Sino
              Si n > segundo y n <> mayor entonces
                 segundo = n
              Fin Si
           FinSi
        Fin Si
     Sino
        Escribir "Ingrese solo numeros positivos"
     FinSi
     Escribir "Ingrese el siguiente numero (0 para terminar)"
     Leer n
  Fin Mientras
  Si contador >= 2 entonces
     Escribir "Mayor = " mayor
     Escribir "Segundo mayor = ", segundo
  sino
     Escribir "Hay menos de dos valores ingresados"
  Fin Si
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

n	Estado de las variables				Salida
7, 3, 9, 5, 0	n = 7 cont = 1 mayor = 7 segundo = 0	n = 3 cont = 2 mayor = 7 segundo = 3	n = 9 cont = 3 mayor = 9 segundo = 7	n = 5 cont = 4 mayor = 9 segundo = 7	Mayor = 9 Segundo mayor: 7
4, 0	n = 0 cont = 1 mayor = 4 segundo = 0	n > 0 Final			Hay menos de dos valores

```c
#include <stdio.h>

int main()
{
        int n, mayor = 0, segundo = 0, contador = 0;

    printf("Ingrese un numero entero positivo (0 para terminar): ");
    scanf("%d", &n);

    while (n != 0) {
        if (n > 0) {
            contador = contador + 1;

            if (contador == 1) {
                mayor = n;
            } else {
                if (n > mayor) {
                    segundo = mayor;
                    mayor = n;
                } else {
                    if (n > segundo && n != mayor) {
                        segundo = n;
                    }
                }
            }
        } else {
            printf("Ingrese solo numeros positivos\n");
        }

        printf("Ingrese el siguiente numero (0 para terminar): ");
        scanf("%d", &n);
    }

    if (contador >= 2) {
        printf("Mayor = %d\n", mayor);
        printf("Segundo mayor = %d\n", segundo);
    } else
        printf("Hay menos de dos valores ingresados\n");
}
```

```
Ingrese un numero entero positivo (0 para terminar): 7
Ingrese el siguiente numero (0 para terminar): 3
Ingrese el siguiente numero (0 para terminar): 9
Ingrese el siguiente numero (0 para terminar): 5
Ingrese el siguiente numero (0 para terminar): 0
Mayor = 9
Segundo mayor = 7

Process returned 0 (0x0)   execution time : 31.184 s
Press any key to continue.
```

Problema 1.20. Adivina el número

Desarrolla un programa que "piense" un número entre 1 y 100 y pida al usuario adivinarlo. Después de cada intento indicar "más alto", "más bajo" o "¡acertaste!" y al final mostrar cuantos intentos se usaron

1. Análisis del problema

Entrada: Intentos sucesivos del usuario

Proceso: Un ciclo compara el intento del usuario con un número secreto fijo, indicando si es "más alto", "más bajo" o "¡acertaste!" mientras incrementa un contador de intentos

Salida: Mensajes que orientan al usuario para que logre adivinar el número

Restricción: Los números deben de estar dentro del rango indicado

¿Cuál es el máximo de intentos necesarios para el rango 1-100 usando búsqueda binaria?

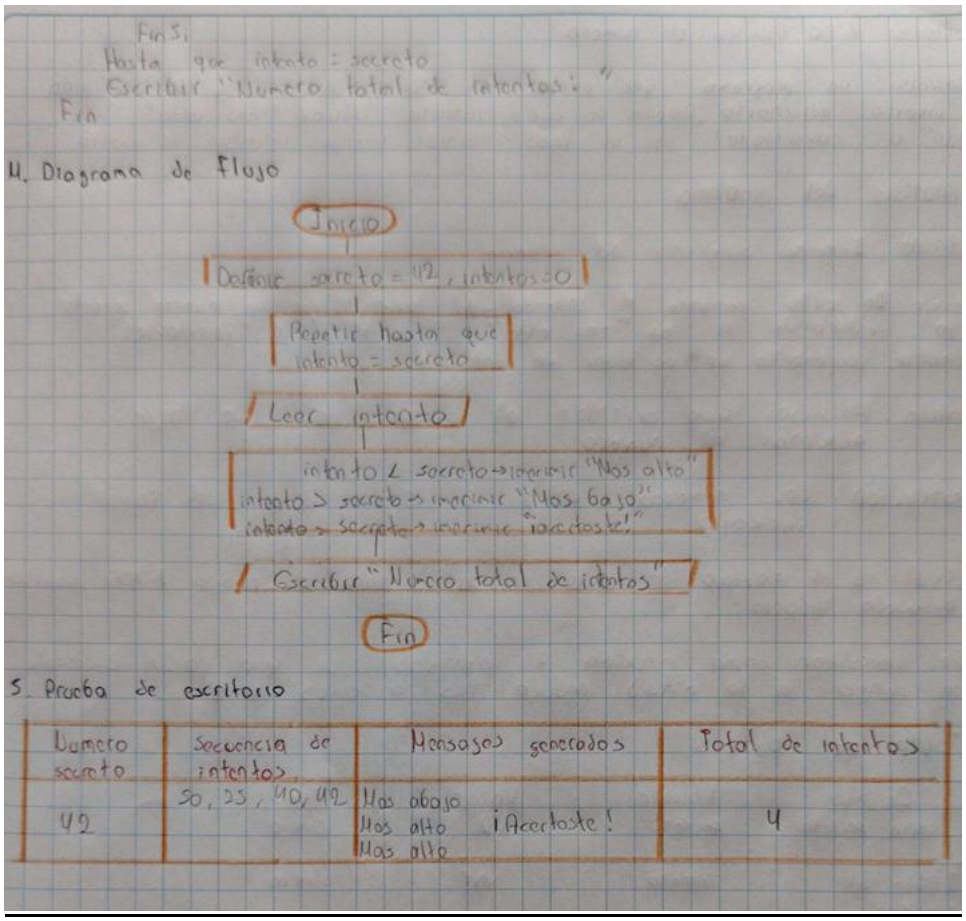
Se necesitan como máximo 7 intentos ya que cada división del rango a la mitad reduce las opciones posibles

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número secreto	secreto	42	entero
Intento del usuario	intento	var	entero
Contador de intentos	intentos	0	entero

3. Pseudocódigo

```
Inicio
  Definir secreto, intento, intentos como enteros
  secreto=0
  intentos=0
  Repetir
    Escribir "Adivina el número (entre 1 y 100)"
    Leer intento
    intentos = intentos + 1
    Si intento < secreto entonces
      Escribir "Más alto"
    Sino
      Si intento > secreto entonces
        Escribir "Más bajo"
      Sino
        Escribir "¡Acertaste!"
      FinSi
```




```c
#include <stdio.h>

int main()
{
        int secreto = 42;
    int intento, intentos = 0;

    do {
        printf("Adivina el numero (entre 1 y 100): ");
        scanf("%d", &intento);
        intentos = intentos + 1;

        if (intento < secreto) {
            printf("Mas alto\n");
        } else {
            if (intento > secreto) {
                printf("Mas bajo\n");
            } else {
                printf("¡Acertaste!\n");
            }
        }
    } while (intento != secreto);

    printf("Numero total de intentos: %d\n", intentos);
}
```

```
Adivina el numero (entre 1 y 100): 10
Mas alto
Adivina el numero (entre 1 y 100): 40
Mas alto
Adivina el numero (entre 1 y 100): 70
Mas bajo
Adivina el numero (entre 1 y 100): 50
Mas bajo
Adivina el numero (entre 1 y 100): 45
Mas bajo
Adivina el numero (entre 1 y 100): 43
Mas bajo
Adivina el numero (entre 1 y 100): 41
Mas alto
Adivina el numero (entre 1 y 100): 42
¡Acertaste!
Numero total de intentos: 8

Process returned 0 (0x0)   execution time : 47.154 s
Press any key to continue.
```